ETH
Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

Improving the Asymptotics of Authenticated Private Information Retrieval with Client-Side Preprocessing

Master Thesis

Domenico Nobile

December 4, 2025

Advisors: Prof. Dr. D. Hofheinz, M. Dietz
Department of Computer Science, ETH Zürich

Abstract

Private Information Retrieval (PIR) schemes enable a client to efficiently obtain data from a database at some index x while hiding information about x. Recent works [6, 19, 10] have studied a family of schemes that deviates slightly from the classical definition of PIR, by adding a preprocessing phase after which the client stores a *hint* that can be used for efficient retrieval of multiple, arbitrarily chosen indices. This allows for the design of schemes with an otherwise unachievable sublinear server runtime. Furthermore, seeking to obtain practical feasibility, the schemes described in these works are only based on One-Way Functions. A parallel line of research on PIR, called Authenticated PIR [5, 18, 7], has developed schemes that guarantee both client privacy and data integrity.

In this work, we investigate the intersection of client-side preprocessing PIR and authenticated PIR. To our knowledge, the only existing protocol in this setting is Wang et al.'s CRUST [18]. We present a new protocol that achieves $\widetilde{\mathcal{O}}_\lambda(\sqrt{n})$ client and server runtime, while incurring in only $\widetilde{\mathcal{O}}_\lambda(n^{1/4})$ bandwidth, where n is the size of the database and $\widetilde{\mathcal{O}}_\lambda(\cdot)$ hides the security parameter λ and polylogarithmic factors. This represents an asymptotic improvement over CRUST, which only achieves $\widetilde{\mathcal{O}}_\lambda(\sqrt{n})$ bandwidth. Our scheme is inspired by techniques from multi-party computation, and builds on the ideas introduced in [10]. Crucially, it only relies on weak and well-studied assumptions, namely, the existence of One-Way Functions and Collision-Resistant Hash Functions.

Contents

Chapter 1

Introduction

Private Information Retrieval (PIR) schemes allow a client to retrieve data from a public database DB stored at one or more servers, without revealing any kind of information to any of the servers involved. A typical example is the retrieval of DNS records: the DNS server's content is public by design, but the client has a clear interest in hiding their query to the server.

To address this problem more thoroughly, we consider the DB as a string of n bits. The client is then interested in learning the value of DB at some index $x \in \{1, \ldots, n\}$. To do this, the client interacts with one or more servers (depending on the protocol specification). Crucially, in a multi-server setting, it is always assumed that the servers do not share information with each other, significantly simplifying the protocol. The property of PIR, henceforth referred to as *privacy*, then implies that the view of each server in its interaction with the client is independent of x.

It should be clear that the problem is definitely solvable: a naive solution would let the client download the entire database. This would clearly hide the query index x, but at the cost of a huge $\mathcal{O}(n)$ overhead in bandwidth, client runtime and server runtime. Thus, what makes PIR interesting is the challenge of reducing these three complexities.

1.1 Classical PIR

Starting from the seminal work of Chor, Goldreich, Kushilevitz, Sudan [3], researchers have made considerable improvements to PIR schemes. In [3], one of the presented results is a 2-server PIR scheme that achieves $\mathcal{O}(n^{1/3})$ bandwidth while still incurring in linear runtime for the servers. In that scheme, the notion of privacy is information-theoretical: each server's view has the exact same distribution for any queried index x. Subsequent works starting from Chor et al. [4] have introduced and studied a weaker privacy

guarantee that is no longer information-theoretic but only computational. This enabled the design of PIR schemes with more efficient communication complexity, and opened to the otherwise impossible single-server setting [12, 2, 9, 8, 16]. In these schemes, both the client and the server(s) run algorithms that are entirely stateless across queries. We will refer to such schemes as belonging to the *classical* PIR setting.

Despite the remarkable improvements in bandwidth, *classical* PIR schemes have one major drawback. As shown by Beimel et al. [1], it is provably impossible to design a PIR protocol in the classical setting that achieves sublinear server computation. In other words, the server runtime is necessarily $\Omega(n)$. The intuition behind this is quite easy to grasp: a sublinear server does not even have the time to read through the whole database, implying that the server learns that the client is definitely not interested in the untouched DB entries.

1.2 Improving Server Runtime: PIR with Preprocessing

To avoid the linear lowerbound on server computation two ideas have been proposed and explored:

- **Server-side preprocessing.** The server(s) encode the database with a public encoding function, and store the encoded database $E(\mathrm{DB})$. The client and the server(s) then execute a protocol on $E(\mathrm{DB})$. This idea is due to Beimel et al. [1], who give an information-theoretic protocol with server-side preprocessing. But this approach comes at the cost of storing an encoded DB which can be orders of magnitude bigger, raising concerns on its practicality. For example, as pointed out in [6], a small 1MB database would be encoded to a huge 1EB codeword just to achieve $n^{0.6}$ server computation and bandwidth in the protocol described in [1]. From this line of research, we also mention Lin et al. [14], which achieves better costs but using heavy-weight assumptions (Ring-LWE).
- **Client-side preprocessing.** The client and server(s) first execute a preprocessing phase, which ends with the client storing a hint. Once the client later selects an index to query, the hint allows a more lightweight query phase with the server. Crucially, the hint is designed to be reusable across many queries, making sublinear server runtime possible because the preprocessing costs may be amortized over many queries.

As we will see later, our work focuses entirely on client-side preprocessing. We also refer to the section on related work for a more comprehensive overview of client-side preprocessing papers.

1.3 Authenticated PIR

Modern client-server protocols increasingly rely on the integrity of the exchanged data. For example, consider the harm a lying DNS server could cause by redirecting the client toward malicious endpoints. For this reason, it is reasonable to assume a malicious server, which may deviate from the protocol at will.

An Authenticated PIR scheme is capable of guaranteeing retrieval of the correct database bit while protecting the client's privacy [5]. Interestingly, construction of a PIR scheme from an integrity-protected scheme and vice versa is a non-trivial problem. To see why, consider the following naive proposal:

> Consider a protocol that satisfies integrity by sending Merkle proofs of inclusion attached to the claimed DB bit, and consider any classical PIR scheme. Then construct a new protocol as follows: the server augments DB to a new database DB' by attaching to each i-th database entry $\mathrm{DB}[i]$ its proof of inclusion π_i, i.e. $\mathrm{DB}'[i] := \mathrm{DB}[i]||\pi_i$. To query for index x, the client uses the PIR protocol to retrieve the desired bit $\mathrm{DB}[x]$, as well as each bit of π_x. The client then uses proof π_x to verify the correctness of the procedure, and accepts or rejects accordingly.

In a malicious server scenario, it makes sense to assume that the server knows whether the client accepted or not: it is not hard to imagine a practical implementation that somehow leaks this information to the server, e.g. by re-running the query after an unusually short amount of time. Surprisingly, this adjustment to the attacker model breaks privacy, even though we used a perfectly working PIR scheme. The reason is subtle: the server may intentionally break some of the proofs of inclusion, but not with the goal of breaking integrity; instead, the server draws conclusions on which areas of the DB queried based on whether the client accepted or not. This kind of attack is known as a *selective-failure* attack.

Although this problem was already mentioned in one of the seminal PIR works [12], most PIR research focused on honest-but-curious servers. However, several works in recent years [5, 18, 7] have shortened the gap between Authenticated PIR and honest-but-curious PIR. We will discuss these in more detail in the section on related work.

1.4 Research Question and Our Contribution

Because incorporating integrity in PIR is inherently challenging, we ask whether it is possible to design an Authenticated PIR protocol from weak

assumptions that benefits from the latest advancements in client-side preprocessing PIR.

Our contribution to this line of research is two-fold:

- We analyze the client-size preprocessing setting with a level of formalism that is considerably higher than prior works.
- We provide a 2-server, authenticated, client-side preprocessing PIR protocol from One Way Functions (OWF) and Collision-Resistant Hash Functions (CRHF) with the following properties:
  - $\widetilde{\mathcal{O}}_\lambda(\sqrt{n})$ client state size
  - $\widetilde{\mathcal{O}}_\lambda(\sqrt{n})$ bandwidth, $\widetilde{\mathcal{O}}_\lambda(n)$ server time and $\widetilde{\mathcal{O}}_\lambda(\sqrt{n})$ client time in the preprocessing phase
  - $\widetilde{\mathcal{O}}_\lambda(\sqrt[4]{n})$ bandwidth, $\widetilde{\mathcal{O}}_\lambda(\sqrt{n})$ server time and $\widetilde{\mathcal{O}}_\lambda(\sqrt{n})$ expected client time for each query in the query phase

  where $\widetilde{\mathcal{O}}_\lambda(\cdot)$ hides the security parameter λ and polylogarithmic factors. If we amortize the complexity of the preprocessing phase, we have $\widetilde{\mathcal{O}}_\lambda(\sqrt[4]{n})$ bandwith and $\widetilde{\mathcal{O}}_\lambda(\sqrt{n})$ client and server time.

Our construction is built from the client-side preprocessing scheme described by Goshal et al. [10], which completely lacks a notion of integrity, thus only assuming semi-honest servers. More specifically, our scheme is private and integrity-protected in a setting where one of the two servers (which we call the *left* server) is fully malicious, whereas the other server (the *right* server), is semi-honest. We do not exclude the existence of protocols with stronger security guarantees. However, we emphasize that in a fully malicious scenario there is no way for a protocol to achieve integrity, as all holders of the DB are not to be trusted.

We also remark that the achieved complexities are quite low, and could be further improved on the hidden factors to make a practical implementation even more viable. Lastly, although [10] is built only on OWFs, we emphasize that our protocol does not rely on complex assumptions that imply public-key cryptography (it only adds the CRHF assumption). This means that practical feasibility only depends on the aforementioned hidden factors.

1.5 Related Work

Starting from the work by Corrigan-Gibbs and Kogan [6], an interesting line of research [19, 13, 10] spanning recent years has brought remarkable improvements in the client-side preprocessing setting. Of these, the work by Goshal et al. [10] (henceforth referred to as GZS) is capable of reducing

bandwidth in the query phase to $\widetilde{\mathcal{O}}_\lambda(\sqrt[4]{n})$, while keeping sublinear server and client computation.

Regarding Authenticated PIR, we highlight two main results. The work by Falk et al. [7] cleverly compiles an Authenticated PIR scheme starting from a blackbox PIR scheme, with no additional assumptions. However, their definition of PIR is incompatible with client-side preprocessing (it uses server-side preprocessing), and as far as we know a generalization to client-side preprocessing, if possible, is not trivial. To our knowledge, the only Authenticated PIR construction that is fully compatible with the client-side preprocessing setting is CRUST [18]. In this paper, the authors give an Authenticated PIR scheme in our attacker model (one server fully-malicious, the other honest-but-curious), only assuming OWF, and that uses client-side preprocessing. However, their result achieves $\widetilde{\mathcal{O}}_\lambda(\sqrt{n})$ bandwidth per query.

Seeking to improve on CRUST's bandwidth, we considered GZS, and noticed that CRUST's approach is incompatible with the new ideas brought in GZS. More specifically, we noticed that applying CRUST to GZS allowed for selective-failure attacks, which could only be prevented by significantly increasing the client's storage. As we will see in Chapter 2, our protocol integrates Authenticated PIR by applying a technique inspired by cut-and-choose, a well-know technique in multi-party computation.

We also mention a few additional constructions in the client-side preprocessing setting [17, 11]. Both of them achieve polylogarithmic bandwidth, but the scheme in [17] assumes LWE, which implies public-key cryptography, and [11] is really inefficient in practice.

1.6 Thesis Outline

This Master's Thesis is structured as follows:

- **Chapter 2 (Technical Overview):** We give an informal yet detailed walkthorugh of the ideas in our construction.
- **Chapter 3 (Notation and Preliminaries):** We write the formal definitions in preparation for *Chapter* 4.
- **Chapter 4 (Full Construction and Proofs):** We define our protocol precisely and prove that it achieves correctness, privacy and integrity according to our definitions.
- **Chapter 5 (Conclusion):** We draw conclusions on our protocol, outline its strengths and its limitations, highlighting future directions for improvement.

Chapter 2

Technical Overview

2.1 Designing PIR schemes with client-side preprocessing

Most 2-server PIR schemes with client-side preprocessing start from an idea that is due to the very first paper on (classical) PIR [3]. Let $\mathrm{DB} \in \{0,1\}^n$, and let $x \in \{1,\ldots,n\}$ be the query index. The client can then draw a subset S_1 of indices uniformly at random. That is, $S_1 \subseteq [n]$. Of course, S_1 may or may not contain index x. S_1 is then sent to the first server, which answers with $p_1 = \bigoplus_{i \in S_1} \mathrm{DB}[i]$, i.e. the XOR of the DB values at the specified indices. We call this the *parity* of S_1. The client then creates a new set S_2 from S_1 by simply adding/removing x, depending on if x was already in S_1 or not. In other words, the symmetric difference between S_1 and S_2 is x. S_2 is sent to the second server, which answers with its parity p_2. It is easy to see that $\mathrm{DB}[x] = p_1 \oplus p_2$. Furthermore, one can easily show that both S_1 and S_2's marginal distributions are completely independent of x.

The idea presented above has one major problem: S_1 and S_2 require $\mathcal{O}(n)$ space on average. Even if we compressed their size by using, say, a PRF, we still could not avoid the $\mathcal{O}(n)$ server computation needed to compute the parities. After all, this is a classical PIR scheme, and as explained in Chapter 1 it necessarily incurs in linear server time. However, this toy protocol has one property that is interesting for our purposes: S_1 is drawn with no dependency on x.

To address the issue with the sizes of S_1 and S_2, one could be tempted to simply reduce their size. However, the smaller the subset, the more likely it is that x is not in S_1, leaking information about x. This leads us to the core idea of client-side preprocessing. Let the two servers be the *right* and the *left* server, and consider the following toy protocol:

Preprocessing.

Sample many sets of indices of size $\sqrt{n}$, and obtain their parities from the right server. This can be done with no dependency on x. Then store each set with its parity in a table T.

Query.

1. To query for index x, find a set S in T such that *it contains* x, and let p be its stored parity.
2. Obtain S' by swapping x with a random index $r \in [n]$. This erases the dependency on x. That is, $S' = S \setminus \{x\} \cup \{r\}$
3. Send S' to the left server and obtain its parity p'.
4. The parities p and p' are then used to reconstruct $\mathrm{DB}[x] = p \oplus (p' \oplus \mathrm{DB}[r])$

The main issue with the idea sketched above is that step 4 cannot be performed by the client unless it has $\mathrm{DB}[r]$. But this can be solved by asking for $\mathrm{DB}[r]$ to the first server in the query phase.

We are already really close to a fully working 2-server client-side preprocessing scheme. Indeed, we can easily observe that the server's views (we assume semi-honest servers for now) are entirely independent of x: the right server sees a bunch of random $\sqrt{n}$-subsets of $[n]$ and a random index $r \in [n]$, while the left server sees an object S' generated from a process that depended on x, but ultimately removed such dependency.

However, the scheme has a few issues. Tackling them one at a time will lead us to a fully working scheme.

Issue 1: *The ideal size of T is so big that step 1 of the query phase (finding elements in T) would incur linear client time, and client storage would be linear too.*
Indeed, assuming that the client is interested in a *random* query x, T must contain more than $\sqrt{n}$ entries. For each of them, checking that the set contains x requires scanning through the whole set. Overall this yields linear client time, which should be avoided at all costs (why should this idea be better then downloading the whole DB?).

To solve issue 1, all schemes in [3, 19, 13, 10] act on the sampling procedure of sets in the preprocessing phase. For $n \in \mathbb{N}$, consider the following process:

- Partition $[n]$ into $\sqrt{n}$ chunks indexed with $1, \ldots, \sqrt{n}$, where chunk i contains the elements $[(i-1) \cdot \sqrt{n} + 1, i \cdot \sqrt{n}]$. [1]
- For each chunk $\ell \in [\sqrt{n}]$, sample a random offset $\delta_\ell \leftarrow_\$ [\sqrt{n}]$
- Output the ordered set $S := \big((i-1) \cdot \sqrt{n} + \delta_i\big)_{i \in [\sqrt{n}]}$

[1] Here, by $[a, b]$ we mean $(a, a+1, \ldots, b)$

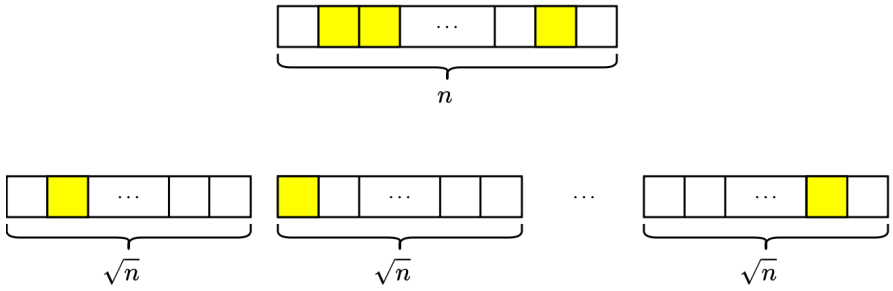


Figure 2.1: The new sampling technique for the sets in T (below) compared to the old approach (above). The yellow squares denote the chosen indices. For example, in the second chunk we take the element of offset 1, which is the $(\sqrt{n}+1)$-th element of the DB.

The process is really intuitive, and it is depicted in Figure 2.1. We call the distribution described by this process $\mathcal{D}_n$. In the following, we will refer to an index as being *in the ℓ-th chunk*, which means that its value is in $[(\ell-1)\cdot\sqrt{n}+1, \ell\cdot\sqrt{n}]$. Lastly, for an index $x \in [n]$, we define its *offset* as its position in a particular chunk (see Figure 2.1).

The new sampling procedure may seem arbitrary, but it allows to reduce both client storage and runtime by using PRFs: on input a key $\mathbf{sk}$ and a chunk index $i \in [\sqrt{n}]$, $\mathsf{PRF}(\mathbf{sk}, i)$ would return a pseudorandom offset $\delta \in [\sqrt{n}]$. The set would then be reconstructed by the server from the succint key $\mathbf{sk}$ as

$$\left\{(i-1)\sqrt{n} + \mathsf{PRF}(\mathbf{sk}, i)\right\}_{i\in[\sqrt{n}]} .$$

Crucially, each set in T can now be stored as a PRF key $\mathbf{sk}$ of size $\widetilde{\mathcal{O}}_\lambda(1)$, and checking if an element x is contained in the set generated by $\mathbf{sk}$ only requires one PRF evaluation $\mathsf{PRF}(\mathbf{sk}, \ell)$ where ℓ is the chunk of x.

This idea, due to Zhou et al's PIANO [19], solves issue 1. But there is a side effect that must be handled. Since the sets sent to the two servers now have a specific structure (one index for each chunk), we need to slightly change our toy protocol. So far, to query for index x the client has to take a set from T that contains x and swap x with a *random* replacement r. To preserve the invariant of the structure of sets drawn from $\mathcal{D}_n$, we cannot take *any* replacement anymore. Instead, we should swap x with a random index that belongs to the same chunk as x. However, this means that we cannot ask for $\mathrm{DB}[r]$ to the right server on the fly: the server would learn the chunk of our hidden index x. The following refinement takes care of this:

Preprocessing.

1. Sample many sets of indices from $\mathcal{D}_n$, and obtain their parities from the right server. Then store each set with its parity in a table T.

2. For each chunk, sample many replacement indices r in that chunk, and obtain their DB value $\mathrm{DB}[r]$ from the first server. Then store each index with its value in a table R.

Queries.

1. To query for index x, find a set S in T such that *it contains* x, and let p its stored parity. Also, find a replacement entry r in R in x's chunk.
2. Obtain S' by swapping x with r. That is, $S' = S \setminus \{x\} \cup \{r\}$
3. Send S' to the left server and obtain its parity p'.
4. The parities p and p' are then used to reconstruct $\mathrm{DB}[x] = p \oplus (p' \oplus \mathrm{DB}[r])$

In other words, we introduce a new preprocessing step, where we sample many replacement entries for each chunk, so that we do not need to ask for them to the right server at each query. We note that in the toy protocol described above we intentionally avoided to use the PRF machinery, as it would significantly clutter the notation.

Issue 2: *The scheme only supports one query.*
Ideally, the client-side preprocessing phase should generate a long-lasting hint, so that this query's complexities get amortized. In our toy protocol, extending to multiple queries by allowing for multiple iterations of steps 1-4 of the "Queries" phase would break privacy. The reason is the following: each time we execute step 1, the set S that we take from T should be deleted after one single usage, otherwise the left server could receive two sets S'_1 and S'_2 that are identical except for one entry, implying that probably $x \notin S'_1 \cap S'_2$. On the other hand, simply removing the used set from T is not enough. The issue here is more subtle: if we remove it, the distribution of table T will be skewed towards sets that do not contain x. For example, if the client queried for index 1 several times, it would become very unlikely that 1 will appear in the sets sent to the left server. Similarly, we need to delete used replacement entries, too.

To address this issue, we make sure that after each query the hint table T keeps the same distribution as at the end of step 1. We do this by refining the protocol as follows:

Preprocessing.

1. Sample many sets of indices from $\mathcal{D}_n$, and obtain their parities from the right server. Then store each set with its parity in a table T.
2. For each chunk, sample many replacement indices r_{right} in that chunk, and obtain their DB value $\mathrm{DB}[r_{\text{right}}]$ from the right server. Then store each index with its value in a table R_{right}.

3. For each chunk, sample many replacement indices r_{left} in that chunk, and obtain their DB value $\text{DB}[r_{\text{left}}]$ from the left server. Then store each index with its value in a table R_{left}.

Query.

1. To query for index x, find a set S in T such that *it contains* x, and let p its stored parity. Also, find a replacement entry r_{right} in R_{right} such that r_{right} is in x's chunk. Remove the chosen entries from T and R_{right}.
2. Obtain S' by swapping x with r_{right}. That is, $S' = S \setminus \{x\} \cup \{r\}$
3. Send S' to the left server and obtain its parity p'.
4. The parities p and p' are then used to reconstruct $\text{DB}[x] = p \oplus (p' \oplus \text{DB}[r_{\text{right}}])$
5. Refresh table T as follows:
   a) Sample a set S_{new} from $\mathcal{D}_n$ conditioned on $x \in S_{\text{new}}$.
   b) Find a replacement entry r_{left} in R_{left} such that r_{left} is in x's chunk and remove it from R_{left}
   c) Obtain S'_{new} by swapping x with r_{left}.
   d) Send S'_{new} to the right server and receive its parity p'_{new}.
   e) Compute the parity of S_{new} $p_{\text{new}} = (p'_{\text{new}} \oplus \text{DB}[r_{\text{left}}]) \oplus \text{DB}[x]$, then store S_{new} with its parity in T.

Remark. To avoid cluttering the description above, we have been slightly imprecise: when looking for entries in T, R_{right} and R_{left}, the client could not be able to find them. This clearly depends on the sizes of these tables. In the full protocol description we describe the optimal sizes to make this event unlikely. But in the unfortunate case that it happens, the client simply aborts. In other words, we lose a negligible factor in correctness.

The protocol above is finally a working client-side preprocessing PIR scheme (PIANO [19]). Let us roughly analyze its efficiency. Each entry in T, R_{right}, R_{left} takes $\widetilde{\mathcal{O}}_\lambda(1)$ space to be stored. In Chapter 4 we will see that if we bound the number of queries that can be made to $\widetilde{\mathcal{O}}_\lambda(\sqrt{n})$, we only need $\widetilde{\mathcal{O}}_\lambda(\sqrt{n})$ entries in T, R_{right} and R_{left}, for a total client storage of $\widetilde{\mathcal{O}}_\lambda(\sqrt{n})$. For bandwidth, PRFs allow a communication in the preprocessing phase that balances upstream and downstream costs ($\widetilde{\mathcal{O}}_\lambda(1)$ for each entry of the tables in the hint). The main issue with bandwidth is in the query phase.

Issue 3: *The scheme requires $\widetilde{\mathcal{O}}_\lambda(\sqrt{n})$ bandwidth per query in the query phase.* When replacing x with a replacement entry, there is no way to succintly send the new set to the server. Indeed, while querying for index x, the PRF key **sk** contains information on x, and there is no trivial way to obtain a new

key that generates the same set except for one specific entry. This causes a rather cumbersome $\widetilde{\mathcal{O}}(\sqrt{n})$ bottleneck in the upstream bandwidth of the query phase.

Issue 3 is the motivation behind the GZS construction, which we now describe.

2.2 The GZS construction

To solve issue 3, what we would like to use is a primitive called "Privately Programmable Pseudorandom Sets". This primitive would ideally allow for generation of succint keys that describe a set from $\mathcal{D}_n$ (perfect for our purposes), but also enable to *program* these keys to obtain keys of relatively small size that describe the exact same set, except for one entry at a given chunk. Given a programmed key $\mathbf{sk}'$, the server should be able to efficiently unpack it and compute the parity of the set it describes.

Unfortunately, to the best of our knowledge, there does not exist a construction of this primitive built from OWFs such that the size of the programmed keys is less than $\sqrt{n}$. But the clever observation in the GZS scheme is that we can construct a somewhat weaker primitive (which they call "Privately Programmable Pseudorandom Sets with List Decoding", or PPPS) from OWFs that works exactly as above, except for the reconstruction of the set from a programmed key $\mathbf{sk}'$: on input such key, an algorithm ListDecode outputs a collection of candidate sets, with the guarantee that exactly one of them is the right one, but with no knowledge of which.

More specifically, for a key $\mathbf{sk}$, the primitive offers the following algorithms:

- $\mathsf{Set}(\mathbf{sk})$ which gives the original set sampled from $\mathcal{D}_n$
- $\mathsf{Program}(\mathbf{sk}, \ell, \delta)$ which gives a programmed key $\mathbf{sk}'$ that describes a set that is identical to $\mathsf{Set}(\mathbf{sk})$, except the ℓ-th element is replaced by $(\ell - 1)\sqrt{n} + \delta$
- $\mathsf{ListDecode}(\mathbf{sk}')$ which gives a tuple of $n^{1/4}$ candidate sets $(S_1, \ldots, S_{n^{1/4}})$
- $\mathsf{Recover}(\mathbf{sk}', \ell)$ which outputs information on which set would be the correct one in the output of ListDecode, given a chunk index $\ell \in [\sqrt{n}]$

Crucially, the size of a programmed key is less succint $\left(\widetilde{\mathcal{O}}_\lambda(\sqrt[4]{n})\right)$, but still less than $\sqrt{n}$, and the algorithms are efficient, in that they do not worsen the asymptotics of PIANO. The key observation is that this approach balances the upstream and downstream bandwidth with a trick that is not new to PIR schemes (see the first PIR paper [3]): we reduce upstream bandwidth by reducing the information provided to the server, and the server tries to "fill the blanks" with a series of guesses, increasing downstream bandwidth. We

refer to Chapter 3 for a proper definition of PPPS, and to the GZS paper [10] for its implementation details.

This idea can be easily applied to our toy protocol (which then becomes identical to the GZS protocol): in the query phase, we compute $\mathbf{sk}' \leftarrow \mathsf{Program}(\mathbf{sk}, \mathsf{chunk}(x), \mathsf{offset}(r))$ where $\mathbf{sk}$ is a key subject to $x \in \mathsf{Set}(\mathbf{sk})$, $\mathsf{chunk}(x)$ returns the chunk where x is, and r is a replacement entry belonging to such chunk. Upon receipt of a programmed key, the left server computes parities on each of the candidate sets output by the ListDecode algorithm. Overall, this yields the desired $\widetilde{\mathcal{O}}_\lambda(\sqrt[4]{n})$ bandwith in the query phase, while keeping $\widetilde{\mathcal{O}}_\lambda(\sqrt{n})$ bandwidth in the preprocessing phase.

2.3 Adding integrity with cut-and-choose

With the goal of designing an *authenticated* PIR scheme, we now consider a different attacker model. More specifically, we assume that one server (the left server) is actively malicious, i.e. it may deviate from the protocol at will. Furthermore, the left server learns whether or not the client aborts the protocol at each query. On the other hand, the right server stays semi-honest.

Inspired by the optimization of the GZS sheme, we tried to apply the newest techniques in Authenticated PIR to GZS, with the goal of obtaining a protocol that achieves the same asymptotic complexities of GZS but in this stronger attacker model. Unfortunately, the only 2-server Authenticated PIR scheme in the client-side preprocessing setting, CRUST [18], led to a selective-failure attack in our case. We will not delve into the details of CRUST, because their techniques are not required by our construction, but for the sake of completeness we say that the issue arises in the ListDecode algorithm: the server could selectively corrupt some of the $n^{1/4}$ answers and learn information on the position of the queried index x.

Preventing selective-failure attacks requires de-correlating the query index from the abort conditions. We know that the GZS protocol aborts if no sets in the hint table T contain x, and a similar dependency on x can be found in the replacement entry tables R_{right} and R_{left}. However, correctness of the scheme ensures that this happens with negligible probability, so we ignore this dependency. Thus, we need to design a protocol that catches a cheating left server with high probability, and no matter the queried index. To do this, we use a technique inspired by cut-and-choose.

Recall that in the query phase of the GZS protocol, the client takes a key $\mathbf{sk}$ from T such that its set $\mathsf{Set}(\mathbf{sk})$ contains x and programs it to obtain $\mathbf{sk}' \leftarrow \mathsf{Program}(\mathbf{sk}, \mathsf{chunk}(x), r_{\text{right}})$. The client then sends $\mathbf{sk}'$ to the left server, which computes $(S_1, \ldots, S_{n^{1/4}}) \leftarrow \mathsf{ListDecode}(\mathbf{sk}')$, and returns parities $p_1, \ldots, p_{n^{1/4}}$, where p_i is the parity of S_i. We would like to be able to

check that each of these parities is correct, but we cannot do that: we could ask the right server since it is assumed to be honest, but this would break privacy of course. However, using the right server as a ground truth is a good intuition.

The key idea is the following: whenever making a query x, sample λ keys $\widetilde{\mathbf{sk}}$ at random. Contrary to the query key $\mathbf{sk}$ taken from T, these keys have no dependency on x. Then, program them at a random chunk $\widetilde{\ell}$ with a random index $\widetilde{r}$, that is, $\widetilde{\mathbf{sk}}' \leftarrow \mathsf{Program}(\widetilde{\mathbf{sk}}, \widetilde{\ell}, \widetilde{r})$, and send these keys to the right server. The right server will then compute the usual $n^{1/4}$ parities and send them to the client. To check that the left server is acting honestly, the client sends the "real" key $\mathbf{sk}$ shuffled inside of the other "fake" keys. Since the "fake" keys (henceforth referred to as verification keys) are indistinguishable from the "real" key (henceforth evaluation key), the left server will need to guess which of the received keys is the evaluation key, and therefore has probability $1/\lambda$ to convince the client of the incorrect DB value.

Since we want exponential instead of polynomial security, we strengthen integrity by increasing the number of evaluation keys to be λ as well. Then, for each evaluation key $\mathbf{sk}_i$, we reconstruct a candidate for $\mathrm{DB}[x]$ y_i as usual, obtaining λ candidates. The reconstructed bit will be the majority one in $\{y_i\}_{i\in[\lambda]}$. A cheating server would then have to guess exactly which of the 2λ keys are verification keys, and corrupt at least half of the remaining λ keys.

We are really close to our final construction now. At the moment, we execute the following protocol:

Preprocessing. Same as in the previous toy protocol.

Queries.

1. To query for index x, find λ keys $(\mathbf{sk}_i)_{i\in[\lambda]}$ in T that unpack to sets that contain x. Also, find replacement entries in R_{right}, and remove the used entries from T and R_{right} as usual.
2. Program the keys with the replacement entries to obtain programmed keys $(\mathbf{sk}'_i)_{i\in[\lambda]}$.
3. Sample λ verification keys $(\widetilde{\mathbf{sk}}_i)_{i\in[\lambda]}$ at random, and program them at a random chunk and with a random index to obtain keys $(\widetilde{\mathbf{sk}}'_i)_{i\in[\lambda]}$.
4. Send the programmed verification keys to the right server, and obtain the reference answers
5. Send the 2λ programmed keys to the left server in a random order, and obtain the server's answers.

6. For each answer associated to a verification key, compare it with the reference answer obtained from the right server, and abort if it is different.
7. If the client did not abort, proceed with the reconstruction using the remaining answers and output the reconstructed bit that appears the most.
8. Refresh the hint table as usual.

Since the verification keys have no dependency on the query, we can sample all of them in the preprocessing phase, and store them in a verification table V. This requires the client to know the number of queries that it will perform upfront. However, this is not an issue: we have a $\widetilde{\mathcal{O}}_\lambda(\sqrt{n})$ bound on the number of queries to guarantee that the client likely finds hint table and replacement table entries, so we can sample that many entries (times λ but this does not affect asymptotics). Hence, the overall size of V is $\widetilde{\mathcal{O}}_\lambda(\sqrt{n})$, keeping the client storage to $\widetilde{\mathcal{O}}_\lambda(\sqrt{n})$.

Issue 4: *Due to the creation of the verification table, the preprocessing phase now requires $\widetilde{\mathcal{O}}_\lambda(n^{3/4})$ bandwidth.*
Indeed, to fetch a single entry of the hint table, the client sends $\widetilde{\mathcal{O}}_\lambda(1)$ bits (the key, the random chunk and the random index), but receives a sequence of $n^{1/4}$ parities.

To solve issue 4, we may apply a very small change: the right server sends a hash of its answer instead of the entire answer. Indeed, the client only needs the right server's answer to compare it with the left server's. This clearly solves issue 4 without increasing runtime.

The last problem to address in our protocol is that the (malicious) left server is also involved in the preprocessing phase: it receives the replacement entries r_{left} and answers with $\text{DB}[r_{\text{left}}]$. These entries are used to refresh the hint table T, as explained earlier. To make sure that the client receives the correct values, we use a vector commitment scheme such as a Merkle proof of inclusion: the left server attaches to its response a proof that the client can verify to make sure that the server does not lie on the database content in this phase. The verification is made possible by the right server, which has previously sent the Merkle root of the DB to the client.

Chapter 3

Notation and Preliminaries

3.1 Notation

For $n \in \mathbb{N}$, we indicate the set $\{1, \ldots, n\}$ as $[n]$.

For $a, b \in \mathbb{N}$, $a \leq b$, we define $[a, b] := \{a, \ldots, b\}$.

We use the symbol $||$ to denote string concatenation.

For $b \in \{0, 1\}$, let $\overline{b} := (1 - b) \bmod 2$.

Ordered sets

We indicate ordered multisets (ordered sets from now on) with $(a_1, \ldots, a_n)$. Given an ordered set S of size n, we indicate the i-th element of S as $S[i]$, for $i \in [n]$. The empty ordered set is $()$. We use the $||$ symbol to denote concatenation: $(a_1, \ldots, a_n) \,||\, (b_1, \ldots, b_n) = (a_1, \ldots, a_n, b_1, \ldots, b_n)$. Given an ordered set $(a_1, \ldots, a_n)$, we indicate the number of occurrences in S of some element x as $\#_x(S)$.

Let $S = (a_1, \ldots, a_n)$, $n \in \mathbb{N}$. For $i \in [n]$, we define

$$S[i \leftarrow x] := (a_1, \ldots, a_{i-1}, x, a_{i+1}, a_n),$$

i.e. the ordered set obtained from S by swapping the i-th element with x, while leaving the other elements unchanged.

Interactive algorithms

We will extensively use the notion of interactive algorithms. Given n interactive algorithms $A_1, \ldots, A_n$, we will write

$$(o_1, \ldots, o_n) \leftarrow \langle A_1(i_1), \ldots, A_n(i_n) \rangle$$

to indicate that each algorithm A_j receives input i_j, interacts with the other algorithms, and eventually produces output o_j. Some outputs may take the value – , meaning that the corresponding algorithms output nothing.

For each algorithm A_j, we also define its view view_{A_j} as the tuple $(i_j, \mathbf{in}_j, \mathbf{out}_j)$, where $\mathbf{in}_j$ is the sequence of messages received from the other algorithms, and $\mathbf{out}_j$ is the sequence of messages sent by A_j to the other algorithms.

Distribution $\mathcal{D}_n$

Let $n \in \mathbb{N}$ be a perfect forth (i.e., $n^{1/4}$ is an integer). We define the distribution $\mathcal{D}_n$ via the following process:

- Partition $[n]$ into $\sqrt{n}$ chunks indexed with $1, \ldots, \sqrt{n}$, where chunk i contains the elements $[(i-1) \cdot \sqrt{n} + 1, i \cdot \sqrt{n}]$
- For each chunk $\ell \in [\sqrt{n}]$, sample a random offset $\delta_\ell \leftarrow_\$ [\sqrt{n}]$.
- Output the ordered set $S := \big((i-1) \cdot \sqrt{n} + \delta_i\big)_{i \in [\sqrt{n}]}$.

For $x \in [n]$, we also define function $\mathsf{chunk}(x)$ that returns the index $\ell \in [\sqrt{n}]$ of the chunk containing x, and function $\mathsf{offset}(x)$ that returns the offset $\delta \in [\sqrt{n}]$ of x in $\mathsf{chunk}(x)$.

Majority function

Let $S = (b_1, \ldots, b_n)$ be an ordered set of bits. Then:

$$\mathsf{majority}(S) = \begin{cases} 0 & \text{if } \#_0(S) \geq \#_1(S) \\ 1 & \text{otherwise} \end{cases}$$

3.2 Definitions

Two-server preprocessing PIR

Definition 3.1 A two-server preprocessing PIR scheme consists of six interactive algorithms $S^{\mathrm{pre}}_{\mathrm{left}}$, $S^{\mathrm{pre}}_{\mathrm{right}}$, C^{pre}, $S^{\mathrm{query}}_{\mathrm{left}}$, $S^{\mathrm{query}}_{\mathrm{right}}$, C^{query}, which can be invoked in two phases:

1. **Preprocessing:** Given $\lambda \in \mathbb{N}$ and $\mathrm{DB} \in \{0,1\}^n$, $C^{\mathrm{pre}}(1^\lambda, 1^n)$ interacts with $S^{\mathrm{pre}}_{\mathrm{left}}(1^\lambda, \mathrm{DB})$ and $S^{\mathrm{pre}}_{\mathrm{right}}(1^\lambda, \mathrm{DB})$ to obtain a hint $H \in \{0,1\}^* \cup \{\perp\}$ as output, while $S^{\mathrm{pre}}_{\mathrm{right}}$ and $S^{\mathrm{pre}}_{\mathrm{left}}$ output nothing. In short, we write:

$$(H, -, -) \leftarrow \langle C^{\mathrm{pre}}(1^\lambda, 1^n), S^{\mathrm{pre}}_{\mathrm{left}}(1^\lambda, \mathrm{DB}), S^{\mathrm{pre}}_{\mathrm{right}}(1^\lambda, \mathrm{DB}) \rangle$$

2. **Queries:** Given $\lambda \in \mathbb{N}$, $x \in [n]$, $\mathrm{DB} \in \{0,1\}^n$ and $H \in \{0,1\}^*$, $C^{\mathrm{query}}(1^\lambda, x, H)$ interacts with $S^{\mathrm{query}}_{\mathrm{left}}(1^\lambda, \mathrm{DB})$ and $S^{\mathrm{query}}_{\mathrm{right}}(1^\lambda, \mathrm{DB})$ to obtain an updated hint H' and a bit $y \in \{0,1,\perp\}$, where $\perp$ indicates that the client aborts. The server algorithms $S^{\mathrm{query}}_{\mathrm{left}}$ and $S^{\mathrm{query}}_{\mathrm{right}}$ output nothing. In short, we write:

$$((H', y), -, -) \leftarrow \langle C^{\mathrm{query}}(1^\lambda, x, H), S^{\mathrm{query}}_{\mathrm{left}}(1^\lambda, \mathrm{DB}), S^{\mathrm{query}}_{\mathrm{right}}(1^\lambda, \mathrm{DB})\rangle$$

Furthermore, the scheme must satisfy the following properties.

- **Correctness.** The following holds with overwhelming probability: for every $x \in [n]$, querying for x to honestly behaving servers results in the client output $\mathrm{DB}[x]$. More formally, for every polynomial P, there exists a negligible function negl such that for every security parameter $\lambda \in \mathbb{N}$, for every database $\mathrm{DB} \in \{0,1\}^n$, where $n \leq P(\lambda)$, for any sequence of $q \leq P(\lambda)$ queries $x_1, \ldots, x_q \in [n]$, in the execution of the PIR scheme with DB and queries $x_1, \ldots, x_q$:

$\mathrm{PIR}(1^\lambda, \mathrm{DB}, (x_1, \ldots, x_q))$:

1. $(H_0, -, -) \leftarrow \langle C^{\mathrm{pre}}(1^\lambda, 1^n),\ S^{\mathrm{pre}}_{\mathrm{left}}(1^\lambda, \mathrm{DB}),\ S^{\mathrm{pre}}_{\mathrm{right}}(1^\lambda, \mathrm{DB})\rangle$
2. For each $t \in [q]$:
   $((H_t, y_t), -, -) \leftarrow \langle C^{\mathrm{query}}(1^\lambda, x_t, H_{t-1}), S^{\mathrm{query}}_{\mathrm{left}}(1^\lambda, \mathrm{DB}), S^{\mathrm{query}}_{\mathrm{right}}(1^\lambda, \mathrm{DB})\rangle$

it holds that $y_t = \mathrm{DB}[x_t]$ for each t with probability at least $1 - \mathsf{negl}(\lambda)$.

- **Left server integrity.** This guarantees that for each queried index x_t, even when the left server behaves maliciously, the client is almost never led to accept an incorrect value, meaning that either the client recovers $\mathrm{DB}[x_t]$ or aborts. More formally, consider the following integrity game for an adversary $\mathcal{A}$ against the PIR scheme:

$\textsc{Integrity}^{\mathcal{A}}(1^\lambda)$:

1. $(\mathrm{DB}, (x_1, \ldots, x_q), \mathsf{st}_{\mathcal{A}}) \leftarrow \mathcal{A}(1^\lambda)$
2. $(H_0, \mathsf{st}_{\mathcal{A}}, -) \leftarrow \langle C^{\mathrm{pre}}(1^\lambda, 1^n), \mathcal{A}(\mathsf{st}_{\mathcal{A}}), S^{\mathrm{pre}}_{\mathrm{right}}(1^\lambda, \mathrm{DB})\rangle$
3. For each $t \in [q]$:
   a) $((H_t, y_t), \mathsf{st}_{\mathcal{A}}, -) \leftarrow \langle C^{\mathrm{query}}(1^\lambda, x_t, H_{t-1}), \mathcal{A}(\mathsf{st}_{\mathcal{A}}), S^{\mathrm{query}}_{\mathrm{right}}(1^\lambda, \mathrm{DB})\rangle$
   b) If $y_t \notin \{\mathrm{DB}[x_t], \perp\}$, output 1
4. Output 0

The advantage of $\mathcal{A}$ against this game is defined as

$$\mathsf{IntAdv}_\lambda(\mathcal{A}) = P[\textsc{Integrity}^{\mathcal{A}}(1^\lambda) = 1]$$

The scheme satisfies left server integrity if for every p.p.t. adversary $\mathcal{A}$ there exists a negligible function negl such that for every $\lambda \in \mathbb{N}$ the advantage $\mathsf{IntAdv}_\lambda(\mathcal{A})$ is at most $\mathsf{negl}(\lambda)$.

- **Privacy.** Privacy requires that the view of each server be independent of the queried index, hence concealing it. We assume that the two servers do not talk to each other in the protocol. This leads to the definition of two notions of privacy, that is, privacy with respect to each individual server: left server and right server privacy.

  For left server privacy, we assume that the left server behaves maliciously. This opens to the possibility of selective-failure attacks. To capture this kind of attack in our notion of privacy, we let the adversary see if the client aborted or not at the end of each query. Formally, a scheme satisfies left server privacy if there exists a probabilistic polynomial time simulator $\mathsf{LeftSim}$ such that for every p.p.t. adversary $\mathcal{A}$ acting as the left server there exists a negligible function negl such that for every $\lambda \in \mathbb{N}$ $\mathcal{A}'s$ advantage

  $$\mathsf{LeftPrivAdv}_\lambda(\mathcal{A}) = |P[\textsc{LeftReal}^{\mathcal{A}}(1^\lambda) = 1] - P[\textsc{LeftIdeal}^{\mathcal{A}}(1^\lambda) = 1]|$$

  is at most $\mathsf{negl}(\lambda)$, where games LeftReal and LeftIdeal are defined as follows:

$\textsc{LeftReal}^{\mathcal{A}}(1^\lambda)$:

1. $(\mathrm{DB}, (x_1, \ldots, x_q), \mathsf{st}_{\mathcal{A}}) \leftarrow \mathcal{A}(1^\lambda)$
2. $(H_0, \mathsf{st}_{\mathcal{A}}, -) \leftarrow \langle C^{\mathrm{pre}}(1^\lambda, 1^n), \mathcal{A}(\mathsf{st}_{\mathcal{A}}), S^{\mathrm{pre}}_{\mathrm{right}}(1^\lambda, \mathrm{DB})\rangle$
3. Let $b_0 \leftarrow \perp$
4. For each $t \in [q]$:
   $((H_t, y_t), \mathsf{st}_{\mathcal{A}}, -) \leftarrow \langle C^{\mathrm{query}}(1^\lambda, x_t, H_{t-1}), \mathcal{A}(\mathsf{st}_{\mathcal{A}}, b_{t-1}), S^{\mathrm{query}}_{\mathrm{right}}(1^\lambda, \mathrm{DB})\rangle$
   and set b_t to 1 if $y_t = \perp$, 0 otherwise.
5. Output bit $b' \leftarrow \mathcal{A}(\mathsf{st}_{\mathcal{A}}, b_q)$

$\textsc{LeftIdeal}^{\mathcal{A}}(1^\lambda)$:

1. $(\mathrm{DB}, (x_1, \ldots, x_q), \mathsf{st}_{\mathcal{A}}) \leftarrow \mathcal{A}(1^\lambda)$
2. $(\mathsf{st}_{\mathsf{Sim}}, \mathsf{st}_{\mathcal{A}}, -) \leftarrow \langle \mathsf{LeftSim}(1^\lambda, 1^n), \mathcal{A}(\mathsf{st}_{\mathcal{A}}), S^{\mathrm{pre}}_{\mathrm{right}}(1^\lambda, \mathrm{DB}) \rangle$
3. Let $b_0 \leftarrow \perp$
4. For each $t \in [q]$:
   $((\mathsf{st}_{\mathsf{Sim}}, y_t), \mathsf{st}_{\mathcal{A}}, -) \leftarrow \langle \mathsf{LeftSim}(\mathsf{st}_{\mathsf{Sim}}), \mathcal{A}(\mathsf{st}_{\mathcal{A}}, b_{t-1}), S^{\mathrm{query}}_{\mathrm{right}}(1^\lambda, \mathrm{DB}) \rangle$
   and set b_t to 1 if $y_t = \perp$, 0 otherwise.
5. Output bit $b' \leftarrow \mathcal{A}(\mathsf{st}_{\mathcal{A}}, b_q)$

For right server privacy, we assume semi-honest behavior of both servers. More specifically, the scheme satisfies right server privacy if there exists a p.p.t. simulator RightSim such that for every p.p.t. adversary $\mathcal{A}$ there exists a negligible function negl such that for every $\lambda \in \mathbb{N}$ $\mathcal{A}$'s advantage

$$\mathsf{RightPrivAdv}_\lambda(\mathcal{A}) = |P[\textsc{RightReal}^{\mathcal{A}}(1^\lambda)] - P[\textsc{RightIdeal}^{\mathcal{A}}(1^\lambda)]|$$

is at most $\mathsf{negl}(\lambda)$, where games RightReal and RightIdeal are defined as follows:

$\textsc{RightReal}^{\mathcal{A}}(1^\lambda)$:

1. $(\mathrm{DB}, (x_1, \ldots, x_q), \mathsf{st}_{\mathcal{A}}) \leftarrow \mathcal{A}(1^\lambda)$
2. $(H_0, -, -) \leftarrow \langle C^{\mathrm{pre}}(1^\lambda, 1^n), S^{\mathrm{pre}}_{\mathrm{left}}(1^\lambda, \mathrm{DB}), S^{\mathrm{pre}}_{\mathrm{right}}(1^\lambda, \mathrm{DB}) \rangle$
3. For each $t \in [q]$:
   $((H_t, y_t), -, -) \leftarrow \langle C^{\mathrm{query}}(1^\lambda, x_t, H_{t-1}), S^{\mathrm{query}}_{\mathrm{left}}(1^\lambda, \mathrm{DB}), S^{\mathrm{query}}_{\mathrm{right}}(1^\lambda, \mathrm{DB}) \rangle$
4. Output bit $b \leftarrow \mathcal{A}(\mathsf{st}_{\mathcal{A}}, \mathsf{view}_{S^{\mathrm{pre}}_{\mathrm{right}}}, \mathsf{view}_{S^{\mathrm{query}}_{\mathrm{right}}})$

$\textsc{RightIdeal}^{\mathcal{A}}(1^\lambda)$:

1. $(\mathrm{DB}, (x_1, \ldots, x_q), \mathsf{st}_{\mathcal{A}}) \leftarrow \mathcal{A}(1^\lambda)$
2. $(\mathsf{st}_{\mathsf{Sim}}, -, -) \leftarrow \langle \mathsf{RightSim}(1^\lambda, 1^n), S^{\mathrm{pre}}_{\mathrm{left}}(1^\lambda, \mathrm{DB}), S^{\mathrm{pre}}_{\mathrm{right}}(1^\lambda, \mathrm{DB}) \rangle$
3. For each $t \in [q]$:
   $((\mathsf{st}_{\mathsf{Sim}}, y_t), -, -) \leftarrow \langle \mathsf{RightSim}(\mathsf{st}_{\mathsf{Sim}}), S^{\mathrm{query}}_{\mathrm{left}}(1^\lambda, \mathrm{DB}), S^{\mathrm{query}}_{\mathrm{right}}(1^\lambda, \mathrm{DB}) \rangle$
4. Output bit $b \leftarrow \mathcal{A}(\mathsf{st}_{\mathcal{A}}, \mathsf{view}_{S^{\mathrm{pre}}_{\mathrm{right}}}, \mathsf{view}_{S^{\mathrm{query}}_{\mathrm{right}}})$

Lastly, we say that the protocol satisfies privacy if it satisfies both left and right server privacy. Notice that this is weaker than assuming that the adversary has control on both servers simultaneously, because we are assuming no interaction between the servers throughout the protocol.

Two-server preprocessing PIR for B random, distinct queries

For our construction, we start with a weaker PIR definition. In this definition, correctness, privacy and integrity of the protocol only come with a few prerequisites:

- The protocol allows for at most B queries, for some $B \in \mathbb{N}$.
- The $q \leq B$ queries must all be *drawn at random from* $[n]$ and *without* replacement, i.e. they must be random and distinct.

Definition 3.2 A two-server preprocessing PIR scheme for B random, distinct queries consists of six interactive algorithms $S^{\text{pre}}_{\text{left}}$, $S^{\text{pre}}_{\text{right}}$, C^{pre}, $S^{\text{query}}_{\text{left}}$, $S^{\text{query}}_{\text{right}}$, C^{query} defined exactly as in Definition 3.1. Furthermore, the scheme must satisfy the usual properties of correctness, left server integrity and privacy, but their definitions are modified as follows:

- **Correctness.** Correctness still holds for any DB, but the query sequence $(x_1, \ldots, x_q)$ is composed of q random, distinct indices, i.e. $x_i \leftarrow_{\$} [n] \setminus \{x_1, \ldots, x_{i-1}\}$ for each $i \in [q]$, drawn at the beginning of the game, for any $q \leq B$.
- **Left server integrity.** The adversary chooses DB and query number $q \leq B$, but has no control on the query sequence $(x_1, \ldots, x_q)$, which is drawn from the aforementioned distribution.
- **Privacy.** Same changes as left server integrity.

Privately Programmable Pseudorandom Set with List Decoding

Definition 3.3 A Privately Programmable Pseudorandom Set with List Decoding (PPPS) is a tuple of the following algorithms:

- $\mathbf{sk} \leftarrow \mathsf{Gen}(1^\lambda, n)$: takes in 1^λ, where $\lambda \in \mathbb{N}$ is a security parameter, and the size of the universe set n, and outputs a secret key $\mathbf{sk}$.
- $S \leftarrow \mathsf{Set}(\mathbf{sk})$: takes in a secret key $\mathbf{sk}$, and returns an ordered set S of size $\sqrt{n}$, such that $S[i] \in [(i-1) \cdot \sqrt{n} + 1, i \cdot \sqrt{n}]$ for each $i \in [\sqrt{n}]$.
- $\mathbf{sk}' \leftarrow \mathsf{Program}(sk, \ell, \delta_\ell)$: takes in a secret key $\mathbf{sk}$, a target chunk $\ell \in [\sqrt{n}]$, a desired offset $\delta_\ell \in [\sqrt{n}]$, and outputs a programmed key $\mathbf{sk}'$.
- $(S_1, \ldots, S_L) \leftarrow \mathsf{ListDecode}(\mathbf{sk}')$: takes in a programmed key $\mathbf{sk}'$ and outputs a list of sets $S_1, \ldots, S_L$.

- $i \leftarrow \mathsf{Recover}(\mathbf{sk}', \ell)$: takes in a programmed key $\mathbf{sk}'$ and a chunk index $\ell \in [\sqrt{n}]$ and outputs an index $i \in [L]$.

Furthermore, a PPPS must satisfy the following properties:

- **Correctness.** For any $\lambda, n \in \mathbb{N}$, for any $\ell, \delta_\ell \in [\sqrt{n}]$, the following holds with probability 1: let $\mathbf{sk} \leftarrow \mathsf{Gen}(1^\lambda, n)$, $\mathbf{sk}' \leftarrow \mathsf{Program}(\mathbf{sk}, \ell, \delta_\ell)$, $(S_1, \ldots, S_L) \leftarrow \mathsf{ListDecode}(\mathbf{sk}')$, and $i \leftarrow \mathsf{Recover}(\mathbf{sk}', \ell)$, then it must hold that $S_i = \mathsf{Set}(\mathbf{sk})[\ell \leftarrow (\ell - 1) \cdot \sqrt{n} + \delta_\ell]$. [1]
- **Pseudorandomness.** The following two distributions are computationally indistinguishable:
  - Sample $S \leftarrow_\$ \mathcal{D}_n$ and output S
  - Sample $\mathbf{sk} \leftarrow \mathsf{Gen}(1^\lambda, n)$, output $\mathsf{Set}(\mathbf{sk})$
- **Private programmability.** We require that for any polynomial P, any p.p.t distinguisher $\mathcal{A}$, there exists a negligible function negl such that for every $\lambda \in \mathbb{N}$, for every $n \leq P(\lambda)$ that is a perfect square, for any $\ell \in [\sqrt{n}]$, for any index x that belongs to the ℓ-th chunk, $\mathcal{A}$'s distinguishing advantage between the games below is at most $\mathsf{negl}(\lambda)$:

  Real. Sample $\mathbf{sk} \leftarrow \mathsf{Gen}(1^\lambda, n)$ subject to $x \in \mathsf{Set}(\mathbf{sk})$, let $\delta_\ell \leftarrow_\$ [\sqrt{n}]$ and let $\mathbf{sk}' \leftarrow \mathsf{Program}(\mathbf{sk}, \ell, \delta_\ell)$, output $\mathbf{sk}'$.

  Ideal. Sample $\widetilde{\mathbf{sk}} \leftarrow \mathsf{Gen}(1^\lambda, n)$, $\tilde{\ell} \leftarrow_\$ [\sqrt{n}], \tilde{\delta} \leftarrow_\$ [\sqrt{n}]$, and let $\widetilde{\mathbf{sk}}' \leftarrow \mathsf{Program}(\widetilde{\mathbf{sk}}, \tilde{\ell}, \tilde{\delta})$. Output $\widetilde{\mathbf{sk}}'$.

Lemma 3.4 *For any $n \in \mathbb{N}$, and assuming the existence of PRFs, there exists a PPPS scheme with the following efficiency properties:*

- $\mathsf{Gen}(\cdot)$ *outputs keys of size $\widetilde{\mathcal{O}}_\lambda(1)$ in $\widetilde{\mathcal{O}}_\lambda(1)$ time*
- $\mathsf{Set}(\cdot)$ *takes $\widetilde{\mathcal{O}}_\lambda(\sqrt{n})$ time to produce its output*
- $\mathsf{Program}(\cdot)$ *takes $\widetilde{\mathcal{O}}_\lambda(\sqrt{n})$ time to produce programmed keys of size $\widetilde{\mathcal{O}}_\lambda(n^{1/4})$*
- $\mathsf{ListDecode}(\cdot)$ *takes $\widetilde{\mathcal{O}}_\lambda(\sqrt{n})$ time to produce its output*
- $\mathsf{Recover}(\cdot)$ *takes $\widetilde{\mathcal{O}}_\lambda(n^{1/4})$ time to produce its output*

Furthermore, for any $x \in [n]$, given a key $\mathbf{sk}$ obtained from Gen, it is possible to check that $x \in \mathsf{Set}(\mathbf{sk})$ in constant time.

Proof See [10] for the detailed construction. Notice that our definition of PPPS slightly differs from the paper's definition in two aspects: we added the Recover algorithm, which is embedded in the paper's Program algorithm, and we considered a stronger definition of private programmability. However, the paper's implementation is clearly privately programmable according to

[1] Correctness does not imply anything about the other sets output by ListDecode

our new definition, because its programmed keys look like random bitstrings. We also remark that the ListDecode algorithm only takes $\widetilde{\mathcal{O}}_\lambda(\sqrt{n})$ despite the size of its output being $\widetilde{\mathcal{O}}_\lambda(n^{3/4})$. This is because we are being slightly informal here: as in the paper, we are assuming that there exists some succint way to write the output of ListDecode, which still enables us to make some computations on these sets. In our case, we need to be able to compute the $n^{1/4}$ parities of these sets. This is possible, as explained in the paper. □

Hash function

Definition 3.5 A Hash Function is a tuple of the following algorithms:

- $pp \leftarrow \mathsf{HashSetup}(1^\lambda)$: takes as input 1^λ, where $\lambda \in \mathbb{N}$ is a security parameter, and outputs some public parameters pp
- $h \leftarrow \mathsf{Eval}(pp, x)$: takes in the public parameters pp and some input $x \in \{0,1\}^*$ and deterministically computes a digest $h \in \{0,1\}^\lambda$

Furthermore, the hash function must satisfy the following property:

> **Collision resistance.** For every p.p.t. adversary $\mathcal{A}$ there exists a negligible function negl such that for every $\lambda \in \mathbb{N}$ it holds that
>
> $$P\left[\begin{array}{c|c} \mathsf{Eval}(pp,x) = \mathsf{Eval}(pp,x') & pp \leftarrow \mathsf{HashSetup}(1^\lambda) \\ \wedge & (x,x') \leftarrow \mathcal{A}(1^\lambda, pp) \\ x \neq x' & \end{array}\right] \leq \mathsf{negl}(\lambda)$$

Vector Commitment Scheme

Definition 3.6 A Vector Commitment Scheme (VC) is a tuple of the following algorithms:

- $pp \leftarrow \mathsf{VCSetup}(1^\lambda)$: on input 1^λ where $\lambda \in \mathbb{N}$ is a security parameter, returns some public parameters pp
- $\sigma \leftarrow \mathsf{Commit}(pp, v)$: on input public parameters pp, a string $v \in \{0,1\}^*$, Commit deterministically computes a commitment $\sigma \in \{0,1\}^\lambda$.
- $\pi \leftarrow \mathsf{Prove}(pp, v, i)$: on input public parameters pp, a string $v \in \{0,1\}^*$ and an index $i \in [n]$, Prove outputs a proof π.
- $b \leftarrow \mathsf{Verify}(pp, \sigma, i, v_i, \pi)$: on input public parameters pp, a commitment σ, an index $i \in [n]$, a bit v_i and a proof π, Verify outputs a bit b.

Furthermore, the scheme must satisfy the following properties:

- **Correctness.** For any $\lambda, n \in \mathbb{N}$, any string $v \in \{0,1\}^n$ and any index $i \in [n]$, the following process outputs 1 with probability 1:

1. $pp \leftarrow \mathsf{VCSetup}(1^\lambda)$
2. $\sigma \leftarrow \mathsf{Commit}(pp, v)$
3. $\pi \leftarrow \mathsf{Prove}(pp, v, i)$
4. Output $\mathsf{Verify}(pp, \sigma, i, v[i], \pi)$

- **Binding.** For every p.p.t. adversary $\mathcal{A}$ there exists a negligible function negl such that for any $\lambda \in \mathbb{N}$ it holds that

$$P\left[\begin{array}{c|c} \mathsf{Verify}(pp, \sigma, i, v[i], \pi) = 1 \\ \wedge \\ \mathsf{Verify}(pp, \sigma, i, v'[i], \pi') = 1 \\ \wedge \\ v \neq v' \end{array} \middle| \begin{array}{c} pp \leftarrow \mathsf{VCSetup}(1^\lambda) \\ (v, v', i, \pi, \pi') \leftarrow \mathcal{A}(1^\lambda, pp) \\ \sigma \leftarrow \mathsf{Commit}(pp, v) \end{array}\right] \leq \mathsf{negl}(n)$$

Lemma 3.7 *Assuming Collision-Resistant Hash Functions, there exists a Vector Commitment Scheme with the following efficiency properties:*

- $\mathsf{VCSetup}(1^\lambda)$ *takes* $\widetilde{\mathcal{O}}_\lambda(1)$ *time to output parameters* pp *of size* $\widetilde{\mathcal{O}}_\lambda(1)$.
- $\mathsf{Commit}(pp, v)$ *takes* $\widetilde{\mathcal{O}}_\lambda(n)$ *time to output a commitment* σ *of size* $\widetilde{\mathcal{O}}_\lambda(1)$, *where* n *is the size of* v.
- $\mathsf{Prove}(pp, v, i)$ *takes* $\widetilde{\mathcal{O}}_\lambda(n)$ *time to output a proof* π *of size* $\widetilde{\mathcal{O}}_\lambda(1)$, *where* n *is the size of* v.
- $\mathsf{Verify}(pp, \sigma, i, v_i, \pi)$ *takes* $\widetilde{\mathcal{O}}_\lambda(1)$ *hash evaluations to produce its output.*

Furthermore, for a vector v *of size* n, *it is possible to execute* Prove *on a batch of* k *indices in* $\widetilde{\mathcal{O}}_\lambda(n + k)$ *time.*

Proof See [15]. □

Chapter 4

Full Construction and Proofs

We now state our result formally.

Theorem 4.1 *Assuming the existence of One-Way Functions and Collision-Resistant Hash Functions, there exists a 2-server Authenticated PIR scheme with client-side preprocessing with the following efficiency properties:*

- $\widetilde{\mathcal{O}}_\lambda(\sqrt{n})$ *client storage*
- $\widetilde{\mathcal{O}}_\lambda(\sqrt{n})$ *bandwidth,* $\widetilde{\mathcal{O}}_\lambda(n)$ *server time and* $\widetilde{\mathcal{O}}_\lambda(\sqrt{n})$ *client time in the preprocessing phase, which takes one communication round with each server*
- $\widetilde{\mathcal{O}}_\lambda(\sqrt[4]{n})$, $\widetilde{\mathcal{O}}_\lambda(\sqrt{n})$ *server time and* $\widetilde{\mathcal{O}}_\lambda(\sqrt{n})$ *expected client time for each query in the query phase, which takes one communication round with each server*

To prove Theorem 4.1, we start by providing a PIR scheme for $\lambda\sqrt{n}$ random, distinct queries, as per Definition 3.2, following the same idea of [19, 10]. The protocol is specified in the next section. We will then prove its efficiency, correctness, privacy and integrity. Lastly, we will informally argue how to use this bounded-query protocol to prove Theorem 4.1.

4.1 Detailed Protocol Description

Preprocessing

1. **Setup.** C^{pre} computes $pp_{\mathsf{H}} \leftarrow \mathsf{HashSetup}(1^\lambda)$, $pp_{\mathsf{VC}} \leftarrow \mathsf{VCSetup}(1^\lambda)$ and sends them to both $S^{\text{pre}}_{\text{right}}$ and $S^{\text{pre}}_{\text{left}}$.
2. **Vector commitment.** $S^{\text{pre}}_{\text{right}}$ computes $\sigma \leftarrow \mathsf{Commit}(pp_{\mathsf{VC}}, \mathrm{DB})$ and sends σ to C^{pre}.
3. **Hint table.** Let $M_1 := 2\lambda\sqrt{n}$. Let $T \leftarrow ()$ denote the client's hint table. Repeat M_1 times:

a) C^{pre} samples a fresh PPPS key $\mathbf{sk} \leftarrow \mathsf{Gen}(1^\lambda)$ and sends it to $S^{\text{pre}}_{\text{right}}$

b) $S^{\text{pre}}_{\text{right}}$ computes parity $p \leftarrow \bigoplus_{j \in \mathsf{Set}(\mathbf{sk})} \text{DB}[j]$ and sends it to C^{pre}

c) C^{pre} updates the hint table $T \leftarrow T \mid\mid \Big((\mathbf{sk}, p)\Big)$

4. **Replacement entries.** Let $R_{\text{right}} \leftarrow ()$ and $R_{\text{left}} \leftarrow ()$ denote the client's right and left replacement entries tables. For each chunk $\ell \in [\sqrt{n}]$, repeat the following $M_2 := 3\lambda^2$ times:

   a) C^{pre} samples a random index $r_{\text{right}} \in [n]$ in chunk ℓ and sends it to $S^{\text{pre}}_{\text{right}}$

   b) $S^{\text{pre}}_{\text{right}}$ sends $\rho_{\text{right}} := \text{DB}[r_{\text{right}}]$ to C^{pre}

   c) C^{pre} updates $R_{\text{right}} \leftarrow R_{\text{right}} \mid\mid \Big((r_{\text{right}}, \rho_{\text{right}})\Big)$

   d) C^{pre} samples a random index $r_{\text{left}} \in [n]$ in chunk ℓ and sends it to $S^{\text{pre}}_{\text{left}}$

   e) $S^{\text{pre}}_{\text{left}}$ computes $\pi \leftarrow \mathsf{Prove}(pp_{\mathsf{VC}}, \text{DB}, r_{\text{left}})$ and sends $(\rho_{\text{left}}, \pi)$ to C^{pre} where $\rho_{\text{left}} := \text{DB}[r_{\text{left}}]$

   f) C^{pre} checks the proof validity by running $\mathsf{Verify}(pp_{\mathsf{VC}}, \sigma, r_{\text{left}}, \rho_{\text{left}}, \pi)$. If the check fails, C^{pre} sets $H \leftarrow \perp$ and the preprocessing phase immediately terminates. Otherwise, C^{pre} updates the left replacement entries $R_{\text{left}} \leftarrow R_{\text{left}} \mid\mid \Big((r_{\text{left}}, \rho_{\text{left}})\Big)$

5. **Verification table.** Let $V \leftarrow ()$ denote the client's verification table. Repeat $\lambda^2\sqrt{n}$ times:

   a) C^{pre} samples a fresh PPPS key $\widetilde{\mathbf{sk}} \leftarrow \mathsf{Gen}(1^\lambda)$, a random chunk $\ell \in [\sqrt{n}]$, and a random offset $\delta \in [\sqrt{n}]$, and sends them to $S^{\text{pre}}_{\text{right}}$

   b) $S^{\text{pre}}_{\text{right}}$ computes $\widetilde{\mathbf{sk}}' \leftarrow \mathsf{Program}(\widetilde{\mathbf{sk}}, \ell, \delta)$, then it decodes the key with $(S_1, \ldots, S_{n^{1/4}}) \leftarrow \mathsf{ListDecode}(\widetilde{\mathbf{sk}}')$. For each S_j output by $\mathsf{ListDecode}$, compute $p_j \leftarrow \bigoplus_{k \in S_j} \text{DB}[k]$. Then, $S^{\text{pre}}_{\text{right}}$ sends $h \leftarrow \mathsf{Eval}(pp_{\mathsf{H}}, p_1 || \ldots || p_{n^{1/4}})$ to C^{pre}

   c) C^{pre} updates the verification table $V \leftarrow V \mid\mid \Big((\widetilde{\mathbf{sk}}', \ell, \delta, h)\Big)$

Lastly, C^{pre} assembles and returns the hint $H := (pp_{\mathsf{H}}, T, R_{\text{right}}, R_{\text{left}}, V)$, ending the preprocessing phase.

Query for index $x \in [n]$

The query phase for each query $x \in [n]$ begins with input to C^{query} x and a hint H. The server algorithms $S^{\text{query}}_{\text{right}}$ and $S^{\text{query}}_{\text{left}}$ only receive the DB as input.

Throughout the rest of the query phase, C^{query} sends programmed PPPS keys to the servers. The two servers' algorithms for each received key are identical and specified at the end of this section.

Step 0: Hint check

If $H = \perp$, C^{query} sets $y \leftarrow \perp$ and $H' \leftarrow \perp$, terminating the query phase for query x.

Step 1: Client Querying

C^{query} parses H as $(pp_{\mathsf{H}}, T, R_{\text{right}}, R_{\text{left}}, V)$. Let $Q = ()$. Then for each $i \in [\lambda]$, C^{query} does the following:

a) Find the first entry $(\mathbf{sk}_i, p_i)$ in the hint table such that $x \in \mathsf{Set}(\mathbf{sk}_i)$. If no such entry exists, set $y \leftarrow \perp$ and $H' \leftarrow \perp$ and terminate the query phase for query x. Otherwise, let s_i be the index in T of such entry. Then set $T \leftarrow T[s_i \leftarrow \perp]$

b) Find the first entry $(r_{\text{right},i}, \rho_{\text{right},i})$ in R_{right} such that $r_{\text{right},i}$ is in the $\mathsf{chunk}(x)$-th chunk and remove it from R_{right}. If no such entry exists, set $y \leftarrow \perp$ and $H' \leftarrow \perp$ and terminate the query phase for query x

c) $\mathbf{sk}'_i \leftarrow \mathsf{Program}(\mathbf{sk}_i, \mathsf{chunk}(x), \mathsf{offset}(r_{\text{right},i}))$

d) Update $Q \leftarrow Q \,||\, (\mathbf{sk}'_i)$

Then for each $i \in [\lambda]$, C^{query} does the following:

a) Take the first entry $(\widetilde{\mathbf{sk}}_i, \ell_i, \delta_i, h_i)$ in the verification table and remove it from V

b) $\widetilde{\mathbf{sk}}'_i \leftarrow \mathsf{Program}(\widetilde{\mathbf{sk}}_i, \ell_i, \delta_i)$

c) Update $Q \leftarrow Q \,||\, (\widetilde{\mathbf{sk}}'_i)$

C^{query} then draws a random permutation $\tau \leftarrow \mathsf{Perm}(2\lambda)$ and sends $Q' \leftarrow \tau(Q)$ to $S^{\text{query}}_{\text{left}}$.

Step 2: Client Reconstructing

After receiving 2λ responses $(a'_1, \ldots, a'_{2\lambda})$, C^{query} unshuffles them by computing $(a_1, \ldots, a_{2\lambda}) \leftarrow \tau^{-1}\big((a'_1, \ldots, a'_{2\lambda})\big)$.

- **Verification.** For each $i \in [\lambda]$: if $\mathsf{Eval}(pp_{\mathsf{H}}, a_{\lambda+i}) \neq h_i$, C^{query} sets $y \leftarrow \perp$ and $H' \leftarrow \perp$ and terminates this query phase. Otherwise C^{query} proceeds with the reconstruction step.
- **Reconstruction.** For each $i \in [\lambda]$, C^{query} parses $a_i := (\beta_1, \ldots, \beta_{n^{1/4}})$ and computes $j_i \leftarrow \mathsf{Recover}(\mathbf{sk}'_i, \mathsf{chunk}(x))$. C^{query} then reconstructs the desired DB bit by setting $y \leftarrow \mathsf{majority}\Big((p_i \oplus \rho_{\text{right},i} \oplus \beta_{j_i})_{i \in [\lambda]}\Big)$.

Step 3: Client Refreshing

For each $i \in [\lambda]$ C^{query} repeats the following:

a) Sample $\mathbf{sk}_{\text{new}} \leftarrow \mathsf{Gen}(1^\lambda)$ such that $x \in \mathsf{Set}(\mathbf{sk}_{\text{new}})$

b) Find the first entry $(r_{\text{left}}, \rho_{\text{left}})$ in R_{left} such that r_{left} is in the $\mathsf{chunk}(x)$-th chunk and remove it from R_{left}

c) $\mathbf{sk}'_{\text{new}} \leftarrow \mathsf{Program}(\mathbf{sk}_{\text{new}}, \mathsf{chunk}(x), \mathsf{offset}(r_{\text{left}}))$, and compute the recovering information $j_{\text{new}} \leftarrow \mathsf{Recover}(\mathbf{sk}'_{\text{new}}, \mathsf{chunk}(x))$

d) Send $\mathbf{sk}'_{\text{new}}$ to $S^{\text{query}}_{\text{right}}$

e) Receive $(\beta_{1,\text{new}}, \ldots, \beta_{n^{1/4},\text{new}})$ from $S^{\text{query}}_{\text{right}}$

f) Set $T \leftarrow T[s_i \leftarrow (\mathbf{sk}_{\text{new}}, \rho_{\text{left}} \oplus \beta_{j_{\text{new}}} \oplus y)]$

Let $H' := (T, R_{\text{right}}, R_{\text{left}}, V)$. C^{query} returns (y, H') and terminates the query phase.

Server Responding (both left and right server)

For each received programmed key $\mathbf{sk}'$, the invoked server does the following:

- Compute $(S_1, \ldots, S_{n^{1/4}}) \leftarrow \mathsf{ListDecode}(\mathbf{sk}')$
- Send $a := (\beta_1, \ldots, \beta_{n^{1/4}})$ to C^{query} where $\beta_i = \bigoplus_{j \in S_i} \mathrm{DB}[j]$

4.2 Efficiency

We start by analyzing the costs in the preprocessing phase, one step at a time:

- **Setup.** The client takes $\widetilde{\mathcal{O}}_\lambda(1)$ time to run this phase, and the public parameters are of size $\widetilde{\mathcal{O}}_\lambda(1)$ as well (see Lemma 3.7).
- **Vector commitment.** The right server computes the $\widetilde{\mathcal{O}}_\lambda(1)$ commitment in $\widetilde{\mathcal{O}}_\lambda(n)$ time due to Lemma 3.7.
- **Hint table.** The following is repeated $\widetilde{\mathcal{O}}_\lambda(\sqrt{n})$ times: the client samples in $\widetilde{\mathcal{O}}_\lambda(1)$ time a key of size $\widetilde{\mathcal{O}}_\lambda(1)$ and sends it to the server, which computes its set's parity in $\widetilde{\mathcal{O}}_\lambda(\sqrt{n})$ time, and sends one parity bit. This step yields an overall $\widetilde{\mathcal{O}}_\lambda(\sqrt{n})$ client time, $\widetilde{\mathcal{O}}_\lambda(n)$ server time and $\widetilde{\mathcal{O}}_\lambda(\sqrt{n})$ bandwidth. We also note that the protocol as described in Section 4.1 requires multiple rounds, but we can let the client sample the keys all at once and receive their respective parities all in one round. The same applies to the remaining steps in the preprocessing phase.
- **Replacement entries.** For the right replacement entries, the costs are obvious: $\widetilde{\mathcal{O}}_\lambda(\sqrt{n})$ client time, bandwidth and server time. For the

left replacements, we have an additional $\widetilde{\mathcal{O}}_\lambda(n)$ server time to execute the Prove algorithm on the whole batch of replacement entries. Each proof has size $\widetilde{\mathcal{O}}_\lambda(1)$ and the client runtime $\widetilde{\mathcal{O}}_\lambda(1)$, hence the overall bandwidth and client time are $\widetilde{\mathcal{O}}_\lambda(\sqrt{n})$, and server time is $\widetilde{\mathcal{O}}_\lambda(n)$.

- **Verification table.** Here, the costs are similar to the hint table step, but the process is repeated λq times, and the server answers with a hash. More specifically, the server receives a key and runs the Program algorithm on it, which takes $\widetilde{\mathcal{O}}_\lambda(\sqrt{n})$ time (see Lemma 3.4). The server then computes the parities on the sets output by ListDecode. As explained in Lemma 3.4, this is possible in $\widetilde{\mathcal{O}}_\lambda(\sqrt{n})$ time. Lastly, the server computes the hashes of the parities. This can be done efficiently. Iterating on λq, and since we assume $q = \lambda\sqrt{n}$, we obtain an overall client time of $\widetilde{\mathcal{O}}_\lambda(\sqrt{n})$, server time of $\widetilde{\mathcal{O}}_\lambda(n)$, and $\widetilde{\mathcal{O}}_\lambda(\sqrt{n})$ bandwidth.

Overall, the preprocessing phase requires $\widetilde{\mathcal{O}}_\lambda(\sqrt{n})$ client runtime, $\widetilde{\mathcal{O}}_\lambda(n)$ server runtime and $\widetilde{\mathcal{O}}_\lambda(\sqrt{n})$ bandwidth. For client storage, note that the client only stores the hint, which is composed of the hash public parameters ($\widetilde{\mathcal{O}}_\lambda(1)$), the hint table ($\widetilde{\mathcal{O}}_\lambda(\sqrt{n})$), the replacement entries (both $\widetilde{\mathcal{O}}_\lambda(\sqrt{n})$) and the verification table ($\widetilde{\mathcal{O}}_\lambda(\sqrt{n})$). Hence, the overall hint size is $\widetilde{\mathcal{O}}_\lambda(\sqrt{n})$.

We now move to the query phase for a single query x. Again, we analyze efficiency in steps:

- The client first looks for keys in the hint table such that their set contains x. By pseudorandomness of the PPPS scheme, this step will require inspection of $\sqrt{n}$ keys in expectation. Furthermore, by Lemma 3.4, checking that x is in $\mathsf{Set}(\mathbf{sk})$ for some $\mathbf{sk}$ only requires constant time. Thus, this step (which is repeated λ times) takes $\widetilde{\mathcal{O}}_\lambda(\sqrt{n})$ expected runtime.
- The client then sends 2λ programmed keys. By Lemma 3.4, each key is of size $\widetilde{\mathcal{O}}_\lambda(n^{1/4})$, for a total bandwidth of $\widetilde{\mathcal{O}}_\lambda(n^{1/4})$. These keys are obtained by running the Program algorithm, which requires $\widetilde{\mathcal{O}}_\lambda(\sqrt{n})$ time. Overall, this step takes $\widetilde{\mathcal{O}}_\lambda(\sqrt{n})$ client time.
- For each of the 2λ keys, the left server computes the usual parities on the sets output by ListDecode. As explained in the preprocessing phase, this step will overall be performed in $\widetilde{\mathcal{O}}_\lambda(\sqrt{n})$ time and $\widetilde{\mathcal{O}}_\lambda(n^{1/4})$ bandwidth. Iteration on the 2λ keys yields yet again $\widetilde{\mathcal{O}}_\lambda(\sqrt{n})$ time.
- The client runs the verification step: for each of the λ verification keys, a hash is computed on the server's response. This requires an overall client time of $\widetilde{\mathcal{O}}_\lambda(n^{1/4})$, as the hash is evaluated on an input of such size.

- The client reconstructs the answer bit y by running a constant number of XOR operations λ times. One of the inputs to the XOR operations is one bit from the server's answer, which must be parsed. Hence, this step costs $\widetilde{\mathcal{O}}_\lambda(n^{1/4})$ time.
- The client and right server then run the refresh phase. This phase has nothing new in terms of efficiency compared to the previous steps.

Thus, we can easily conclude that the query phase requires $\widetilde{\mathcal{O}}_\lambda(\sqrt{n})$ client and server time, while incurring in only $\widetilde{\mathcal{O}}_\lambda(n^{1/4})$ bandwidth. Combining this with the amortized server cost yields the efficiency claims in Theorem 4.1.

4.3 Proofs

4.3.1 Main Lemma

We start by proving a lemma that will be helpful in correctness, privacy and integrity. The main claim of this lemma is that we can decorrelate queries. More specifically, the main challenge of this proof is that each query affects the next one via the hint table, which gets refreshed each time. Without this step, it would be hard to make claims on privacy, because each key $\mathbf{sk}$ extracted from the hint table T depends on query index x_t, but we cannot swap it with another key independent of x_t and argue that the adversary would not notice this, because of a chain of dependencies on $\mathbf{sk}$ that we need to clean first. In the end, the lemma shows that the hint table is less complex than it seems, and the process of querying looks identical to a process in which the client samples keys on the fly instead of searching them in the hint table. Furthermore, the lemma proves that a left server acting maliciously in the preprocessing phase has negligible advantage due to the binding property of the commitment scheme. Lastly, it proves that the client will find entries in the T, R_{right} and R_{left} with overwhelming probability, which is essential for correctness.

Lemma 4.2 *Given a distinguisher $\mathcal{D}$, consider the two games $\textsc{Game}_1^{\mathcal{D}}$ and $\textsc{Game}_2^{\mathcal{D}}$ below. For any p.p.t distinguisher $\mathcal{D}$[1], there exists a negligible function* negl *such that for any $\lambda \in \mathbb{N}$ it holds that $\mathcal{D}$'s advantage*

$$|P[\textsc{Game}_1^{\mathcal{D}}(1^\lambda) = 1] - P[\textsc{Game}_2^{\mathcal{D}}(1^\lambda) = 1]|$$

is bounded by $\mathsf{negl}(\lambda)$.

[1] In the games, $\mathcal{D}$ is only allowed to output a q that is bounded by $\lambda\sqrt{n}$

$\textsc{Game}_1^{\mathcal{D}}(1^\lambda)$:

1. $(\mathsf{st}_{\mathcal{D}}, \mathrm{DB}, q) \leftarrow \mathcal{D}(1^\lambda)$, then draw $(x_1, \ldots, x_q)$ from $[n]$ at random without replacement
2. $pp_{\mathsf{VC}} \leftarrow \mathsf{VCSetup}(1^\lambda)$, $\sigma \leftarrow \mathsf{Commit}(pp_{\mathsf{VC}}, \mathrm{DB})$
3. Let $M_1 := 2\lambda\sqrt{n}$, $T \leftarrow ()$. Repeat M_1 times:

   (a) $\mathbf{sk} \leftarrow \mathsf{Gen}(1^\lambda)$ (b) $p := \bigoplus_{j \in \mathsf{Set}(\mathbf{sk})} \mathrm{DB}[j]$ (c) $T \leftarrow T \,||\, \Big((\mathbf{sk}, p)\Big)$
4. Let $M_2 := 3\lambda^2$, $R_{\text{left}} \leftarrow ()$, $R_{\text{right}} \leftarrow ()$. For each $\ell \in [\sqrt{n}]$, repeat M_2 times:
   a) Sample random $r_{\text{right}} \in [n]$ in chunk ℓ. Let $\rho_{\text{right}} := \mathrm{DB}[r_{\text{right}}]$
   b) $R_{\text{right}} \leftarrow R_{\text{right}} \,||\, \Big((r_{\text{right}}, \rho_{\text{right}})\Big)$
   c) Sample random $r_{\text{left}} \in [n]$ in chunk ℓ
   d) $((\rho_{\text{left}}, \pi), \mathsf{st}_{\mathcal{D}}) \leftarrow \mathcal{D}(pp_{\mathsf{VC}}, r_{\text{left}}, \mathsf{st}_{\mathcal{D}})$
   e) $b \leftarrow \mathsf{Verify}(pp_{\mathsf{VC}}, \sigma, r_{\text{left}}, \rho_{\text{left}}, \pi)$. If $b = 0$, $H \leftarrow \perp$. Otherwise, $R_{\text{left}} \leftarrow R_{\text{left}} \,||\, \Big((r_{\text{left}}, \rho_{\text{left}})\Big)$

For $t \in [q]$:

1. If $H = \perp$, set $\mathsf{st}_{\mathcal{D}} \leftarrow \mathcal{D}(\mathsf{st}_{\mathcal{D}}, \perp)$ and skip this iteration.
2. For each $i \in [\lambda]$, find the first entry $(\mathbf{sk}_i, p_i) \in T$ such that $x_t \in \mathsf{Set}(\mathbf{sk}_i)$. If no such entry exists, set $H \leftarrow \perp$, $\mathsf{st}_{\mathcal{D}} \leftarrow \mathcal{D}(\mathsf{st}_{\mathcal{D}}, \perp)$ and skip this iteration. Otherwise let this entry be the s_i-th. Then set $T \leftarrow T[s_i \leftarrow \perp]$
3. Find the first $(r_{\text{right},i}, \rho_{\text{right},i}) \in R_{\text{right}}$ such that $r_{\text{right},i}$ is in the $\mathsf{chunk}(x_t)$-th chunk and remove it from R_{right}. If no such entry exists, set $H \leftarrow \perp$ and skip this iteration.
4. $\mathbf{sk}'_i \leftarrow \mathsf{Program}(\mathbf{sk}_i, \mathsf{chunk}(x_t), \mathsf{offset}(r_{\text{right},i}))$
5. $\mathsf{st}_{\mathcal{D}} \leftarrow \mathcal{D}(\{\mathbf{sk}'_i\}_{i \in [\lambda]}, \mathsf{st}_{\mathcal{D}})$
6. For each $i \in [\lambda]$:
   a) $\mathbf{sk}_{\text{new}} \leftarrow \mathsf{Gen}(1^\lambda)$ such that $x_t \in \mathsf{Set}(\mathbf{sk}_{\text{new}})$.
   b) Find the first entry $(r_{\text{left}}, \rho_{\text{left}}) \in R_{\text{left}}$ such that r_{left} is in the $\mathsf{chunk}(x_t)$-th chunk and remove it from R_{left}.
   c) $\mathbf{sk}'_{\text{new}} \leftarrow \mathsf{Program}(\mathbf{sk}_{\text{new}}, \mathsf{chunk}(x_t), \mathsf{offset}(r_{\text{left}}))$, then obtain $j_{\text{new}} \leftarrow \mathsf{Recover}(\mathbf{sk}'_{\text{new}}, \mathsf{chunk}(x_t))$.
   d) $(S_1, \ldots, S_{n^{1/4}}) \leftarrow \mathsf{ListDecode}(\mathbf{sk}'_{\text{new}})$, $\beta \leftarrow \bigoplus_{i \in S_{j_{\text{new}}}} \mathrm{DB}[i]$
   e) $T \leftarrow T[s_i \leftarrow (\mathbf{sk}_{\text{new}}, \rho_{\text{left}} \oplus \beta \oplus \mathrm{DB}[x_t])]$

Output $\mathcal{D}(\mathsf{st}_{\mathcal{D}})$

$\textsc{Game}_2^{\mathcal{D}}(1^\lambda)$:

1. $(\mathsf{st}_{\mathcal{D}}, \mathrm{DB}, q) \leftarrow \mathcal{D}(1^\lambda)$, then draw $(x_1, \ldots, x_q)$ from $[n]$ at random without replacement
2. $pp_{\mathsf{VC}} \leftarrow \mathsf{VCSetup}(1^\lambda)$, $\sigma \leftarrow \mathsf{Commit}(pp_{\mathsf{VC}}, \mathrm{DB})$
3. Let $M_2 := 3\lambda^2$. For each $\ell \in [\sqrt{n}]$, repeat M_2 times:
   a) Sample random $r_{\text{left}} \in [n]$ in chunk ℓ
   b) $((\rho_{\text{left}}, \pi), \mathsf{st}_{\mathcal{D}}) \leftarrow \mathcal{D}(pp_{\mathsf{VC}}, r_{\text{left}}, \mathsf{st}_{\mathcal{D}})$
   c) $b \leftarrow \mathsf{Verify}(pp_{\mathsf{VC}}, \sigma, r_{\text{left}}, \rho_{\text{left}}, \pi)$. If $b = 0$, $H \leftarrow \perp$.

For $t \in [q]$:

1. If $H = \perp$, set $\mathsf{st}_{\mathcal{D}} \leftarrow \mathcal{D}(\mathsf{st}_{\mathcal{D}}, \perp)$ and skip this iteration.
2. For each $i \in [\lambda]$, sample $\mathbf{sk}_i \leftarrow \mathsf{Gen}(1^\lambda)$ conditioned on $x_t \in \mathsf{Set}(\mathbf{sk}_i)$.
3. Sample $r \leftarrow_{\$} [\sqrt{n}]$
4. $\mathbf{sk}_i' \leftarrow \mathsf{Program}(\mathbf{sk}_i, \mathsf{chunk}(x_t), r)$
5. $\mathsf{st}_{\mathcal{D}} \leftarrow \mathcal{D}(\{\mathbf{sk}_i'\}_{i\in[\lambda]}, \mathsf{st}_{\mathcal{D}})$

Output $\mathcal{D}(\mathsf{st}_{\mathcal{D}})$

Proof We prove the lemma via a few hybrid hops. Consider $\textsc{LemmaHyb}_1$ below (differences are highlighted in yellow).

$\textsc{LemmaHyb}_1^{\mathcal{D}}(1^\lambda)$:

For steps 1-4, the only change is that R_{left} now only stores the indices r_{left}, but not their value ρ_{left}.

For $t \in [q]$:

1–5. Same as in $\textsc{Game}_1$

6. For each $i \in [\lambda]$:
   a) $\mathbf{sk}_{\text{new}} \leftarrow \mathsf{Gen}(1^\lambda)$ such that $x_t \in \mathsf{Set}(\mathbf{sk}_{\text{new}})$.
   b) Find the first entry $r_{\text{left}} \in R_{\text{left}}$ such that r_{left} is in the $\mathsf{chunk}(x_t)$-th chunk and remove it from R_{left}.
   c) $\mathbf{sk}_{\text{new}}' \leftarrow \mathsf{Program}(\mathbf{sk}_{\text{new}}, \mathsf{chunk}(x_t), \mathsf{offset}(r_{\text{left}}))$, then obtain $j_{\text{new}} \leftarrow \mathsf{Recover}(\mathbf{sk}_{\text{new}}', \mathsf{chunk}(x_t))$.
   d) $(S_1, \ldots, S_{n^{1/4}}) \leftarrow \mathsf{ListDecode}(\mathbf{sk}_{\text{new}}')$, $\beta \leftarrow \bigoplus_{i\in S_{j_{\text{new}}}} \mathrm{DB}[i]$
   e) $T \leftarrow T[s_i \leftarrow (\mathbf{sk}_{\text{new}}, \mathrm{DB}[r_{\text{left}}] \oplus \beta \oplus \mathrm{DB}[x_t])]$

Output $\mathcal{D}(\mathsf{st}_{\mathcal{D}})$

This game is identical to $\textsc{Game}_1$, except that we do not use ρ_{left} to compute the parity for $\mathbf{sk}_{\text{new}}$. Instead, we compute $\text{DB}[r_{\text{left}}]$ on the fly. An immediate consequence of this change is that since the ρ_{left} are never used now, we only store indices in R_{left}. This hop essentially proves that the malicious left server has negligible probability of successfully lying about the replacement entries in the preprocessing phase.

We prove the indistinguishability between $\textsc{Game}_1$ and $\textsc{LemmaHyb}_1$ by reducing it to the binding property of the underlying vector commitment. More formally, indistinguishability comes from indistinguishability in a sequence of intermediate hops between $\textsc{Game}_1$ and $\textsc{LemmaHyb}_1$, in which we gradually swap ρ_{left} with $\text{DB}[r_{\text{left}}]$ one at a time, for each of the $\sqrt{n} \cdot M_2$ entries of R_{left}, for a total of $\sqrt{n} \cdot M_2$ hops. Two adjacent hops in this sequence only differ in the event that $\rho_{\text{left}} \neq \text{DB}[r_{\text{left}}]$ and $\mathsf{Verify}(pp_{\mathsf{VC}}, \sigma, r_{\text{left}}, \rho_{\text{left}}, \pi)$. If

$$P\left[\rho_{\text{left}} = \overline{\text{DB}[r_{\text{left}}]} \ \wedge \ \mathsf{Verify}(pp_{\mathsf{VC}}, \sigma, r_{\text{left}}, \rho_{\text{left}}, \pi) = 1\right] \tag{4.1}$$

is negligible, then the games are indistinguishable. The reduction is trivial: on input $(1^\lambda, pp_{\text{VC}})$, the reduction $\mathcal{A}$ sets v to the DB chosen by $\mathcal{D}$. $\mathcal{A}$ then proceeds by simulating $\mathsf{LemmaHyb}_2$ to $\mathcal{D}$. When $\mathcal{A}$ sends r_{left} and receives a proof π' and a bit ρ_{left} from $\mathcal{D}$, $\mathcal{A}$ sets v' to be $\text{DB}[r_{\text{left}} \leftarrow \overline{\text{DB}[r_{\text{left}}]}]$. Then, $\pi \leftarrow \mathsf{Prove}(pp_{\text{VC}}, \text{DB}, r_{\text{left}})$. Lastly, $\mathcal{A}$ outputs $(v, v', r_{\text{left}}, \pi, \pi')$ to the binding challenger. It should be clear that if the condition in (4.1) is true, then $\mathcal{A}$ wins. This concludes the reduction.

Now, consider the next hybrid $\textsc{LemmaHyb}_2$. Since the malicious left server has no control over the replacement entries in $\textsc{LemmaHyb}_1$, we simplify their sampling procedure by removing the replacement entry tables altogether: each replacement entry is sampled on the fly, whenever it is needed. However, games $\textsc{LemmaHyb}_1$ and $\textsc{LemmaHyb}_2$ are not perfectly indistinguishable: in $\textsc{LemmaHyb}_1$ we handle the possibility of not having a replacement entry for a specific chunk by aborting the query, while $\textsc{LemmaHyb}_2$ this cannot happen. Hence, the distinguishability advantage between these two games is bounded by the probability that a failure in the replacement entry table occurs. We now show that this probability is negligible, which proves that $\textsc{LemmaHyb}_1$ and $\textsc{LemmaHyb}_2$ are indistinguishable.

$\textsc{LemmaHyb}_2^{\mathcal{D}}(1^\lambda)$:

Steps 1-4 are identical to $\textsc{LemmaHyb}_1$, but we have no R_{left} and R_{right}, hence we also skip steps 4a, 4b and the update of R_{left}.

For $t \in [q]$:

1. If $H = \perp$, set $\text{st}_{\mathcal{D}} \leftarrow \mathcal{D}(\text{st}_{\mathcal{D}}, \perp)$ and skip this iteration.
2. For each $i \in [\lambda]$, find the first entry $(\mathbf{sk}_i, p_i) \in T$ such that $x_t \in \text{Set}(\mathbf{sk}_i)$. If no such entry exists, set $H \leftarrow \perp$, $\text{st}_{\mathcal{D}} \leftarrow \mathcal{D}(\text{st}_{\mathcal{D}}, \perp)$ and skip this iteration. Otherwise let this entry be the s_i-th. Then set $T \leftarrow T[s_i \leftarrow \perp]$
3. Sample $r \leftarrow_\$ [\sqrt{n}]$
4. $\mathbf{sk}_i' \leftarrow \text{Program}(\mathbf{sk}_i, \text{chunk}(x_t), r)$
5. $\text{st}_{\mathcal{D}} \leftarrow \mathcal{D}(\{\mathbf{sk}_i'\}_{i\in[\lambda]}, \text{st}_{\mathcal{D}})$
6. For each $i \in [\lambda]$:
   a) $\mathbf{sk}_{\text{new}} \leftarrow \text{Gen}(1^\lambda)$ such that $x_t \in \text{Set}(\mathbf{sk}_{\text{new}})$.
   b) Sample $r \leftarrow_\$ [\sqrt{n}]$
   c) $\mathbf{sk}'_{\text{new}} \leftarrow \text{Program}(\mathbf{sk}_{\text{new}}, \text{chunk}(x_t), r)$, then compute $j_{\text{new}} \leftarrow \text{Recover}(\mathbf{sk}'_{\text{new}}, \text{chunk}(x_t))$.
   d) $(S_1, \ldots, S_{n^{1/4}}) \leftarrow \text{ListDecode}(\mathbf{sk}'_{\text{new}})$, $\beta \leftarrow \bigoplus_{i\in S_{j_{\text{new}}}} \text{DB}[i]$
   e) $T \leftarrow T[s_i \leftarrow (\mathbf{sk}_{\text{new}}, \text{DB}[r] \oplus \beta \oplus \text{DB}[x_t])]$

Output $\mathcal{D}(\text{st}_{\mathcal{D}})$

A failure occurs if more than M_2/λ queried indices lie in the same chunk. Let $Y_{t,i}$ be the random variable that is 1 if the t-th query index lies in the i-th chunk, 0 otherwise. Let $X_i = Y_{1,i} + \cdots + Y_{q,i}$. Then it holds that $E[Y_{t,i}] = \frac{1}{\sqrt{n}}$, and $E[X_i] = \frac{\lambda\sqrt{n}}{\sqrt{n}} = \lambda$. Also, recall that we are considering random, non-duplicate queries. This means that $Y_{1,i}, \ldots, Y_{q,i}$ are negatively correlated. Taking the Chernoff bound for negatively correlated variables, we get:

$$P[X_i \geq M_2/\lambda] = P[X_i \geq (1+2)\lambda] \leq e^{\frac{-2^2}{2+2}\lambda} = e^{-\lambda}$$

which is negligible in λ. Taking the union bound over all chunks yields again a probability negligible in λ.

Now consider hybrid $\textsc{LemmaHyb}_3$ below.

$\textsc{LemmaHyb}_3^{\mathcal{D}}(1^\lambda)$:

Steps 1-4 are identical to $\textsc{LemmaHyb}_2$

For $t \in [q]$:

1–5. Same as in $\textsc{Game}_1$

6. For each $i \in [\lambda]$:
   a) $\mathbf{sk}_{\text{new}} \leftarrow \mathsf{Gen}(1^\lambda)$ such that $x_t \in \mathsf{Set}(\mathbf{sk}_{\text{new}})$.
   b) $T \leftarrow T[s_i \leftarrow (\mathbf{sk}_{\text{new}}, \bigoplus_{k \in \mathsf{Set}(\mathbf{sk}_{\text{new}})} \mathrm{DB}[k])]$

Output $\mathcal{D}(\mathrm{st}_{\mathcal{D}})$

This game is perfectly indistinguishable from the previous one, due to correctness of the PPPS scheme: since we do not rely anymore on $\mathcal{D}$'s answers in the preprocessing phase ($\textsc{LemmaHyb}_1$), we can simplify the process of computing the parity for the new entry of the hint table T by directly computing the parity of $\mathsf{Set}(\mathbf{sk}_{\text{new}})$.

Now consider $\textsc{LemmaHyb}_4$ below.

$\textsc{LemmaHyb}_4^{\mathcal{D}}(1^\lambda)$:

1–2. Identical to $\textsc{LemmaHyb}_3$

3. Let $M_1 := 2\lambda\sqrt{n}$, $T \leftarrow ()$. Sample $\mathbf{s} := (s_1, \ldots, s_\lambda)$. If $\mathbf{s} = \perp$, set $H \leftarrow \perp$ and skip this step. For $i \in [M_1]$:
   a) If $i \in \{s_1, \ldots, s_\lambda\}$, sample $\mathbf{sk} \leftarrow \mathsf{Gen}(1^\lambda)$ such that $x_1 \in \mathsf{Set}(\mathbf{sk})$
   b) Else if $i < s_\lambda$, sample $\mathbf{sk} \leftarrow \mathsf{Gen}(1^\lambda)$ such that $x_1 \notin \mathsf{Set}(\mathbf{sk})$
   c) Else sample $\mathbf{sk} \leftarrow \mathsf{Gen}(1^\lambda)$
   d) $p \leftarrow \bigoplus_{j \in \mathsf{Set}(\mathbf{sk})} \mathrm{DB}[j]$
   e) $T \leftarrow T \,||\, \big((\mathbf{sk}, p)\big)$
4. Identical to $\textsc{LemmaHyb}_3$

For $t \in [q]$:

1–6. As in $\textsc{LemmaHyb}_3$

Output $\mathcal{D}(\mathrm{st}_{\mathcal{D}})$

In our construction, T is sampled by repeatedly and independently generating keys with the Gen PPPS algorithm. Let $\mathbf{s} := (s_1, \ldots, s_\lambda)$ be a vector of indices in $[M_1]$ such that $0 < s_1 < \ldots < s_\lambda \leq M_1$. Then let $\alpha_{\mathbf{s}}$ be the probability that $\mathbf{sk}_{s_i}$ is the i-th key in T such that $x_1 \in \mathsf{Set}(x_1)$, for each $i \in [\lambda]$. Then let $\alpha_\perp := 1 - \sum_{\mathbf{s}} \alpha_{\mathbf{s}}$ denote the probability that there are not enough keys satisfying such requirement. In $\textsc{LemmaHyb}_4$ we sample T using a different process. We first sample $\mathbf{s}$ according to the distribution defined above, and then assemble T according to $\mathbf{s}$. The joint distribution of T is identical in both processes by definition of the distribution of $\mathbf{s}$, making the two games identical.

Now consider $\textsc{LemmaHyb}_5$. The only change in this game is that we force $\mathbf{s} \neq \perp$. Hence, the distinguishability of games $\textsc{LemmaHyb}_4$ and $\textsc{LemmaHyb}_5$ is bounded by α_λ. We will now show that α_λ is negligible for our choice of M_1.

$\textsc{LemmaHyb}_5^{\mathcal{D}}(1^\lambda)$:

1–2. Identical to $\textsc{LemmaHyb}_3$

3. Identical to $\textsc{LemmaHyb}_3$, but $\mathbf{s}$ is now sampled conditioned on $\mathbf{s} \neq \perp$.

4. Identical to $\textsc{LemmaHyb}_3$

For $t \in [q]$:

1–6. As in $\textsc{LemmaHyb}_3$, but at step 2 we do not check for a hint table fail.

Output $\mathcal{D}(\mathsf{st}_{\mathcal{D}})$

We begin by proving that the failure probability is negligible in the case of M_1 hint sets $T_{\mathcal{D}_n} := (S_1, \ldots, S_{M_1})$ sampled from $\mathcal{D}_n$ as defined in Section 3.1.

To avoid cluttering the notation, we will refer to index x_1 as x. Let $Y_{x,i}$ be the random variable that is 1 if $x \in S_i$, and let $X_x = Y_{x,1} + \ldots + Y_{x,M_1}$. Then, $E[X_x] = M_1/\sqrt{n} = 2\lambda$. We can then use the Chernoff bound on X_x:

$$P[X_x \leq \lambda] \leq e^{-\frac{2\lambda}{8}} = e^{-\lambda/4}$$

which is negligible in λ.

Now consider the actual sampling in our construction. Each of the M_1 sets will be sampled as $\mathsf{Set}(\mathbf{sk})$ for some randomly sampled $\mathbf{sk} \leftarrow_\$ \mathsf{Gen}(1^\lambda)$. We prove that this change only increases the failure probability by an amount negligible in λ. Let $E_x(S_1, \ldots, S_{M_1})$ be the event that there is no λ-subset in $\{S_1, \ldots, S_{M_1}\}$ of sets containing x. Let $\overline{S_i} := \mathsf{Set}(\mathbf{sk}_i)$, for some $\mathbf{sk}_i \leftarrow \mathsf{Gen}(1^\lambda)$. We are interested in bounding

$$\begin{aligned}\alpha_\perp &= P\left[E_x(\overline{S}_1,\ldots,\overline{S}_{M_1})\right] \\ &= P\left[E_x(\overline{S}_1,\ldots,\overline{S}_{M_1})\right] \\ &\quad - P[E_x(S_1,\ldots,S_{M_1})] \\ &\quad + P[E_x(S_1,\ldots,S_{M_1})]\,.\end{aligned}$$

Let $p_i := P[E_x(\overline{S_1},\ldots,\overline{S_i},S_{i+1},\ldots,S_{M_1})]$, for $i = 0,\ldots,M_1$. Then:

$$p_{M_1} = (p_{M_1} - p_{M_1-1}) + (p_{M_1-1} - p_{M_1-2}) + \ldots + (p_1 - p_0) + p_0$$

We can bound $p_i - p_{i-1}$ by reducing it to the pseudorandomness of the PPPS. Consider the following adversary $\mathcal{A}_i$ against the pseudorandomness of the PPPS: on input a set S, $\mathcal{A}_i$ draws $M_1 - i - 1$ sets $S_{i+1},\ldots,S_{M_1} \leftarrow_\$ \mathcal{D}_n$ and $i-1$ PPPS keys $\mathbf{sk}_1,\ldots,\mathbf{sk}_{i-1}$. Let $\overline{S_j} := \mathsf{Set}(\mathbf{sk}_j)$. Then the adversary counts the occurrences of x among the sets $\overline{S_1},\ldots,\overline{S_{i-1}},S,S_{i+1},\ldots,S_{M_1}$, and outputs 1 if the occurrences are less than λ, 0 otherwise. Clearly, the advantage of the adversary is $p_i - p_{i-1}$.

This makes $\alpha_\perp$ negligible in λ, making the two games indistinguishable.

In the following games LemmaHyb$_6$ and LemmaHyb$_7$, we take advantage of this new sampling process for T: we first sample λ keys conditioned on containing x_1, program them, send the programmed keys to the distinguisher $\mathcal{D}$, and then sample $\mathbf{s}$ and the remaining keys. We still proceed as usuall for all remaining queries $x_2,\ldots,x_q$. For LemmaHyb$_7$, we use the fact that the new keys $\mathbf{sk}_{\text{new}}$ are sampled in exactly the same way as the query keys. This means that we can reverse the hop from LemmaHyb$_3$ to LemmaHyb$_4$, that is, we assemble T in the usual way defined in the construction, but T is now created after the λ query keys were given to the adversary!

Now that table T is sampled exactly as it was sampled before query for x_1, we may repeat the LemmaHyb$_3$–LemmaHyb$_7$ hops for all q queries. This yields the final game Game$_2$. Since q is polynomially bounded in λ, the overall distinguish advantage is still negligible. □

$\textsc{LemmaHyb}_6^{\mathcal{D}}(1^\lambda)$:

1–2. Identical to $\textsc{LemmaHyb}_5$.

3. Identical to step 4 in $\textsc{LemmaHyb}_5$. Step 3 is skipped.

For $t = 1$:

1. If $H = \perp$, set $\mathsf{st}_{\mathcal{D}} \leftarrow \mathcal{D}(\mathsf{st}_{\mathcal{D}}, \perp)$ and skip this iteration.
2. For each $i \in [\lambda]$, sample $\mathbf{sk}_i \leftarrow \mathsf{Gen}(1^\lambda)$ conditioned on $x_1 \in \mathsf{Set}(\mathbf{sk}_i)$.
3. Sample $r \leftarrow_\$ [\sqrt{n}]$
4. $\mathbf{sk}_i' \leftarrow \mathsf{Program}(\mathbf{sk}_i, \mathsf{chunk}(x_t), r)$
5. $\mathsf{st}_{\mathcal{D}} \leftarrow \mathcal{D}(\{\mathbf{sk}_i'\}_{i\in[\lambda]}, \mathsf{st}_{\mathcal{D}})$
6. Let $M_1 := 2\lambda\sqrt{n}$, $T \leftarrow ()$. Sample $\mathbf{s} := (s_1, \ldots, s_\lambda)$ conditioned on $\mathbf{s} \neq \perp$.
   Then for each $i \in [M_1]$:
   a) If $i \in \{s_1, \ldots, s_\lambda\}$, $T \leftarrow T \,||\, (\perp)$ and jump to next iteration
   b) Else if $i < s_\lambda$, sample $\mathbf{sk} \leftarrow \mathsf{Gen}(1^\lambda)$ conditioned on $x_1 \notin \mathsf{Set}(\mathbf{sk})$
   c) Else sample $\mathbf{sk} \leftarrow \mathsf{Gen}(1^\lambda)$
   d) $p \leftarrow \bigoplus_{k\in\mathsf{Set}(\mathbf{sk})} \mathrm{DB}[k]$
   e) $T \leftarrow T \,||\, \Big((\mathbf{sk}, p)\Big)$
7. For each $i \in [\lambda]$:
   a) $\mathbf{sk}_{\text{new}} \leftarrow \mathsf{Gen}(1^\lambda)$ such that $x_t \in \mathsf{Set}(\mathbf{sk}_{\text{new}})$.
   b) $T \leftarrow T[s_i \leftarrow (\mathbf{sk}_{\text{new}}, \bigoplus_{k\in\mathsf{Set}(\mathbf{sk}_{\text{new}})} \mathrm{DB}[k])]$

For $t = 2, \ldots, q$: steps 1–6 are identical to $\textsc{LemmaHyb}_5$.

Output $\mathcal{D}(\mathsf{st}_{\mathcal{D}})$

$\textsc{LemmaHyb}_7^{\mathcal{D}}(1^\lambda)$:

1–3. Identical to $\textsc{LemmaHyb}_6$.

For $t = 1$:

1–5. Identical to $\textsc{LemmaHyb}_6$.

6. Let $M_1 := 2\lambda\sqrt{n}$, $T \leftarrow ()$. Repeat M_1 times:
   a) $\mathbf{sk} \leftarrow \mathsf{Gen}(1^\lambda)$
   b) $p := \bigoplus_{j\in\mathsf{Set}(\mathbf{sk})} \mathrm{DB}[j]$
   c) $T \leftarrow T \cup \Big((\mathbf{sk}, p)\Big)$

For $t = 2, \ldots, q$: steps 1–6 identical to $\textsc{LemmaHyb}_6$.

Output $\mathcal{D}(\mathsf{st}_{\mathcal{D}})$

4.3.2 Correctness for $\lambda\sqrt{n}$ random, distinct queries

Proof Correctness follows directly from Lemma 4.2. Indeed, one of the implications of Lemma 4.2 is that the failure probability coming from a missing replacement entry and that coming from a missing hint table entry are both negligible. Via a trivial reduction to Lemma 4.2, we may jump to a game where the challenger executes the whole protocol (there is no adversary in correctness, because both servers are honest). In this game, one may easily observe that the retrieved bit y_t is always correct: y_t is obtained as $\mathsf{parity}(S) \oplus (\mathsf{parity}(S[x \leftarrow r]) \oplus \mathrm{DB}(r))$, where S is $\mathsf{Set}(\mathbf{sk})$ (which must contain x, and parity is defined as $\mathsf{parity}(S) = \bigoplus_{k \in S} \mathrm{DB}(k)$. A similar observation may be done for the hint refreshing phase. This concludes correctness.

4.3.3 Left server integrity for $\lambda\sqrt{n}$ random, distinct queries

Proof The main idea for proving left server integrity is that the server has no idea which of the 2λ queries will be used as test queries. To be successful, the left server will need to correctly identify the λ test queries, answer honestly to them, and then corrupt the answers to most of the remaining queries. As we will see later, in a scenario where the test queries and the evaluation queries are information theoretically indistinguishable, this game is almost certainly lost by the cheating server. However, the queries in our protocol actually come from different distributions. We prove that this difference gives little additional advantage to an integrity adversary, hence we still achieve integrity. We prove this with a hybrid argument.

Consider $\textsc{IntHyb}_1$ below. For clarity, we will split the games into their preprocessing and query parts.

$\textsc{IntHyb}_1^{\mathcal{A}}(1^\lambda)$:

Preprocessing.

1. $(\mathsf{st}_{\mathcal{A}}, \mathrm{DB}, q) \leftarrow \mathcal{A}(1^\lambda)$, then draw $(x_1, \ldots, x_q)$ at random without replacement
2. $pp_{\mathsf{H}} \leftarrow \mathsf{HashSetup}(1^\lambda)$, $pp_{\mathsf{VC}} \leftarrow \mathsf{VCSetup}(1^\lambda)$, $\sigma \leftarrow \mathsf{Commit}(pp_{\mathsf{VC}}, \mathrm{DB})$
3. Let $M_1 := 2\lambda\sqrt{n}$, $T \leftarrow ()$. Repeat M_1 times:
   (a) $\mathbf{sk} \leftarrow \mathsf{Gen}(1^\lambda)$ (b) $p := \bigoplus_{j \in \mathsf{Set}(\mathbf{sk})} \mathrm{DB}[j]$ (c) $T \leftarrow T \,||\, \Big((\mathbf{sk}, p)\Big)$
4. Let $M_2 := 3\lambda^2$, $R_{\text{left}} \leftarrow ()$, $R_{\text{right}} \leftarrow ()$. For each $\ell \in [\sqrt{n}]$, repeat M_2 times:
   a) Sample random $r_{\text{right}} \in [n]$ in chunk ℓ. Let $\rho_{\text{right}} := \mathrm{DB}[r_{\text{right}}]$
   b) $R_{\text{right}} \leftarrow R_{\text{right}} \,||\, \Big((r_{\text{right}}, \rho_{\text{right}})\Big)$
   c) Sample random $r_{\text{left}} \in [n]$ in chunk ℓ
   d) $((\rho_{\text{left}}, \pi), \mathsf{st}_{\mathcal{A}}) \leftarrow \mathcal{A}(\mathsf{st}_{\mathcal{A}}, pp_{\mathsf{H}}, pp_{\mathsf{VC}}, r_{\text{left}})$
   e) $b \leftarrow \mathsf{Verify}(pp_{\mathsf{VC}}, \sigma, r_{\text{left}}, \rho_{\text{left}}, \pi)$. If $b = 0$, $H \leftarrow \perp$ and skip step 4. Otherwise, $R_{\text{left}} \leftarrow R_{\text{left}} \,||\, \Big((r_{\text{left}}, \rho_{\text{left}})\Big)$

$\textsc{IntHyb}_1^{\mathcal{A}}(1^\lambda)$:

Queries. For $t \in [q]$:

1. If $H = \perp$, set $y_t \leftarrow \perp$ and skip this iteration.
2. Let $Q = ()$. For each $i \in [\lambda]$:
   a) Find the first entry $(\mathbf{sk}_i, p_i) \in T$ such that $x_t \in \mathsf{Set}(\mathbf{sk}_i)$. If no such entry exists, set $y_t \leftarrow \perp$, $H \leftarrow \perp$ and skip this iteration. Otherwise, let this entry be the s_i-th, and set $T \leftarrow T[s_i \leftarrow \perp]$
   b) Find the first $(r_{\text{right},i}, \rho_{\text{right},i}) \in R_{\text{right}}$ such that $r_{\text{right},i}$ is in the $\mathsf{chunk}(x_t)$-th chunk and remove it from R_{right}. If no such entry exists, set $y_t \leftarrow \perp$, $H \leftarrow \perp$ and skip this iteration.
   c) $\mathbf{sk}'_i \leftarrow \mathsf{Program}(\mathbf{sk}_i, \mathsf{chunk}(x_t), \mathsf{offset}(r_{\text{right},i}))$. Update $Q \leftarrow Q \,||\, (\mathbf{sk}'_i)$.
3. For $i \in [\lambda]$:
   a) $\widetilde{\mathbf{sk}}_i \leftarrow \mathsf{Gen}(1^\lambda)$, $\ell_i \leftarrow_\$ [\sqrt{n}]$, $\delta_i \leftarrow_\$ [\sqrt{n}]$
   b) $\widetilde{\mathbf{sk}}'_i \leftarrow \mathsf{Program}(\widetilde{\mathbf{sk}}_i, \ell_i, \delta_i)$
   c) $(S_1, \ldots, S_{n^{1/4}}) \leftarrow \mathsf{ListDecode}(\widetilde{\mathbf{sk}}'_i)$
   d) For each S_j output by ListDecode, set $\widetilde{p}_j \leftarrow \bigoplus_{k \in S_j} \mathrm{DB}[k]$
   e) $h_i \leftarrow \mathsf{Eval}(pp_{\mathsf{H}}, \widetilde{p}_1 || \ldots || \widetilde{p}_{n^{1/4}})$
   f) Update $Q \leftarrow Q \,||\, (\widetilde{\mathbf{sk}}'_i)$.
4. $\tau \leftarrow_\$ \mathsf{Perm}(2\lambda)$, $(\mathsf{st}_{\mathcal{A}}, (a'_1, \ldots, a'_{2\lambda})) \leftarrow \mathcal{A}(\tau(Q), \mathsf{st}_{\mathcal{A}})$, $(a_1, \ldots, a_{2\lambda}) \leftarrow \tau^{-1}\big((a'_1, \ldots, a'_{2\lambda})\big)$.
5. For each $i \in [\lambda]$: if $\mathsf{Eval}(pp_{\mathsf{H}}, a_{\lambda+i}) \neq h_i$, set $y_t \leftarrow \perp$ and $H \leftarrow \perp$ and skip this iteration.
6. For each $i \in [\lambda]$, parse $a_i := (\beta_1, \ldots, \beta_{n^{1/4}})$ and then compute $j_i \leftarrow \mathsf{Recover}(\mathbf{sk}'_i, \mathsf{chunk}(x_t))$.
7. Set $y_t \leftarrow \mathsf{majority}\Big((p_i \oplus \rho_{\text{right},i} \oplus \beta_{j_i})_{i \in [\lambda]}\Big)$. If $y_t \notin \{\perp, \mathrm{DB}[x_t]\}$ output 1
8. Repeat the following λ times:
   a) $\mathbf{sk}_{\text{new}} \leftarrow \mathsf{Gen}(1^\lambda)$ such that $x_t \in \mathsf{Set}(\mathbf{sk}_{\text{new}})$.
   b) Find the first $(r_{\text{left}}, \rho_{\text{left}}) \in R_{\text{left}}$ such that r_{left} is in the $\mathsf{chunk}(x_t)$-th chunk and remove it from R_{left}.
   c) $\mathbf{sk}'_{\text{new}} \leftarrow \mathsf{Program}(\mathbf{sk}_{\text{new}}, \mathsf{chunk}(x_t), \mathsf{offset}(r_{\text{left}}))$, and then compute $j_{\text{new}} \leftarrow \mathsf{Recover}(\mathbf{sk}'_{\text{new}}, \mathsf{chunk}(x_t))$
   d) $(S_1, \ldots, S_{n^{1/4}}) \leftarrow \mathsf{ListDecode}(\mathbf{sk}'_{\text{new}})$, $\beta \leftarrow \bigoplus_{i \in S_{j_{\text{new}}}} \mathrm{DB}[i]$
   e) $T \leftarrow T[s_i \leftarrow (\mathbf{sk}_{\text{new}}, \rho_{\text{left}} \oplus \beta \oplus \mathrm{DB}[x_t])]$

Output 0

In this first hybrid, the challenger acts both as the client and the right server. The only difference with the real protocol is that the verification table is no longer used. Instead, the challenger samples verification table entries on the fly. This is possible because there is no particular requirement that must be satisfied for a verification table entry to be chosen, and we have just enough

entries to make $q \leq \lambda\sqrt{n}$ queries: each query in our scheme consumes λ elements in the verification table, and we have $\lambda \cdot \lambda\sqrt{n}$ entries. Hence, the two games are perfectly indistinguishable.

Now consider IntHyb_2 below. In this game, we change how steps 6 and 7 reconstruct bit y_t. Specifically, for each evaluation key $\mathbf{sk}_i$ taken from T, we use correctness of the PPPS scheme to retrieve the parity of $\mathsf{Set}(\mathbf{sk}_i)$ from its programmed version $\mathbf{sk}'_i$. Indeed, $\mathsf{Set}(\mathbf{sk}_i)$ is identical to one of the sets output by $\mathsf{ListDecode}(\mathbf{sk}'_i)$, S_{j_i}, except for one element, $S_{j_i}[\mathsf{chunk}(x_t)]$. PPPS correctness hence guarantees that p_i obtained as in IntHyb_2 is identical to the parity of $\mathsf{Set}(\mathbf{sk}_i)$. Thus, the two games are perfectly indistinguishable.

$\text{IntHyb}_2^{\mathcal{A}}(1^\lambda)$:

Preprocessing. Identical to IntHyb_1.

Queries. For $t \in [q]$:

1–5. Identical to IntHyb_1.

6. For each $i \in [\lambda]$, parse $a_i := (\beta_1, \ldots, \beta_{n^{1/4}})$ then compute index $j_i \leftarrow \mathsf{Recover}(\mathbf{sk}'_i, \mathsf{chunk}(x_t))$, $(S_1, \ldots, S_{n^{1/4}}) \leftarrow \mathsf{ListDecode}(\mathbf{sk}'_i)$, then compute $p_i \leftarrow \left(\bigoplus_{k \in S_{j_i}} \mathrm{DB}[k]\right) \oplus S_{j_i}[\mathsf{chunk}(x_t)] \oplus \mathrm{DB}[x_t]$

7. Set $y_t \leftarrow \mathsf{majority}\left((p_i \oplus S_{j_i}[\mathsf{chunk}(x_t)] \oplus \beta_{j_i})_{i \in [\lambda]}\right)$. If $y_t \notin \{\perp, \mathrm{DB}[x_t]\}$ then output 1

8. Identical to IntHyb_1.

Output 0

Now consider IntHyb_3 below. This hybrid looks quite different from IntHyb_2, but its indistinguishability from the previous game comes directly from Lemma 4.2. Indeed, we may reduce their indistinguishability to the indistinguishability implied by Lemma 4.2. A lemma distinguisher $\mathcal{D}$ for the lemma games can easily simulate the two integrity hybrids to the distinguisher between IntHyb_2 and IntHyb_3. On input $pp_{\mathsf{VC}}, r_{\text{left}}$, $\mathcal{D}$ simply forwards these to adversary $\mathcal{A}$. Then, on input λ programmed keys, $\mathcal{D}$ computes λ verification keys like in the integrity games, shuffles the 2λ keys and forwards them to the integrity adversary. Then, $\mathcal{D}$ proceeds with the reconstruction of y_t. This is a crucial step in the reduction, as it justifies the hop from IntHyb_1 to IntHyb_2: since $\mathcal{D}$ is only given programmed keys by the lemma challenger, it must be able to reconstruct y_t from the programmed keys only (with the use of the $\mathsf{Recover}$ algorithm, which was not present in the original PPPS definition [10]).

This proves indistinguishability between IntHyb_2 and IntHyb_3.

$\text{IntHyb}_3^{\mathcal{A}}(1^\lambda)$:

Preprocessing. Identical to IntHyb_2, but step 3 is skipped and R_{right} and R_{left} are not used anymore.

Queries. For $t \in [q]$:

1. Identical to IntHyb_2.
2. Let $Q = ()$. For each $i \in [\lambda]$:
   a) Sample $\mathbf{sk}_i \leftarrow \mathsf{Gen}(1^\lambda)$ conditioned on $x_t \in \mathsf{Set}(\mathbf{sk}_i)$
   b) Sample $r \leftarrow_\$ [\sqrt{n}]$
   c) $\mathbf{sk}_i' \leftarrow \mathsf{Program}(\mathbf{sk}_i, \mathsf{chunk}(x_t), r)$
   d) Update $Q \leftarrow Q \,||\, (\mathbf{sk}_i')$.

3–7. Identical to LemmaHyb_2. There is no step 8.

Output 0

Now consider IntHyb_4 below. This hop is a direct application of the private programmability property of the PPPS. The reduction is trivial, and again it is guaranteed by our ability to reconstruct y_t from a programmed key, as the PPPS challenger will only provide a programmed key. As for Lemma 4.2, we are being slightly informal here, as a proper hybrid argument with reduction to private programmability would require a series of intermediate hops, where we apply this change gradually over all q queries. The bound on q ensures that the overall loss will still be negligible in λ.

$\text{IntHyb}_4^{\mathcal{A}}(1^\lambda)$:

Preprocessing. Identical to IntHyb_3.

Queries. For $t \in [q]$:

1. Identical to IntHyb_3.
2. Let $Q = ()$. For each $i \in [\lambda]$:
   a) $\mathbf{sk}_i \leftarrow \mathsf{Gen}(1^\lambda)$
   b) Sample $r \leftarrow_\$ [n]$
   c) $\mathbf{sk}_i' \leftarrow \mathsf{Program}(\mathbf{sk}_i, \mathsf{chunk}(r), \mathsf{offset}(r))$
   d) Update $Q \leftarrow Q \,||\, (\mathbf{sk}_i')$.

3–7. Identical to IntHyb_3.

Output 0

In IntHyb_4 we finally achieved our objective: the 2λ keys are now identically distributed. At this point, we can draw them in a single procedure, as shown in IntHyb_5 below. The challenger then only decides which of the 2λ keys will be used for verification *after* the adversary has answered.

$\text{IntHyb}_5^{\mathcal{A}}(1^\lambda)$:

Preprocessing. Identical to IntHyb_4.

Queries. For $t \in [q]$:

1. If $H = \perp$, set $y_t \leftarrow \perp$ and skip this iteration.
2. Let $Q \leftarrow ()$, For $i \in [2\lambda]$:
   a) $\mathbf{sk}_i \leftarrow \mathsf{Gen}(1^\lambda)$, $r \leftarrow_\$ [n]$
   b) $\mathbf{sk}'_i \leftarrow \mathsf{Program}(\mathbf{sk}_i, \mathsf{chunk}(r), \mathsf{offset}(r))$. Update $Q \leftarrow Q \,||\, (\mathbf{sk}'_i)$.
   c) Update $Q \leftarrow Q \,||\, (\mathbf{sk}'_i)$.
3. $\tau \leftarrow_\$ \mathsf{Perm}(2\lambda)$, then $(\mathsf{st}_{\mathcal{A}}, (a'_1, \ldots, a'_{2\lambda})) \leftarrow \mathcal{A}(Q, \mathsf{st}_{\mathcal{A}})$, $(a_1, \ldots, a_{2\lambda}) \leftarrow \tau((a'_1, \ldots, a'_{2\lambda}))$. Parse $\tau(Q)$ as $(\mathbf{sk}'_1, \ldots, \mathbf{sk}'_{2\lambda})$.
4. For each $i \in [\lambda]$:
   a) Compute h_i from $\mathbf{sk}'_{\lambda+i}$ as done in previous hybrids.
   b) If $\mathsf{Eval}(pp_{\mathsf{H}}, a_{\lambda+i}) \neq h_i$, set $y_t \leftarrow \perp$ and $H \leftarrow \perp$ and skip this iteration.
5. For each $i \in [\lambda]$, parse $a_i := (\beta_1, \ldots, \beta_{n^{1/4}})$, then compute index $j_i \leftarrow \mathsf{Recover}(\mathbf{sk}'_i, \mathsf{chunk}(x_t))$, then $(S_1, \ldots, S_{n^{1/4}}) \leftarrow \mathsf{ListDecode}(\mathbf{sk}'_i)$, $p_i \leftarrow \left(\bigoplus_{k \in S_{j_i}} \mathrm{DB}[k]\right) \oplus S_{j_i}[\mathsf{chunk}(x_t)] \oplus \mathrm{DB}[x_t]$
6. Set $y_t \leftarrow \mathsf{majority}\left((p_i \oplus S_{j_i}[\mathsf{chunk}(x_t)] \oplus \beta_{j_i})_{i \in [\lambda]}\right)$. If $y_t \notin \{\perp, \mathrm{DB}[x_t]\}$ output 1

Output 0

Consider now the last hop to IntHyb_6 below. Here, we change the verification procedure by checking the correctness of the answer directly, instead of comparing its hash. The indistinguishability of these two games is directly implied by the collision resistance of the hash function. The reduction[2] is similar to the reduction to vector commitment binding we performed in Lemma 4.2. The distinguishability advantage between the two games is bounded by the probability that the adversary provides an answer to the challenger that is not $(p_1 || \ldots || p_{n^{1/4}})$, but has the same hash. For the reduction, on input $(1^\lambda, pp_{\mathsf{H}})$, the collision-resistance adverary $\mathcal{A}$ begins the

[2]We are slightly informal here: as in the proof of Lemma 4.2, we actually have several intermediate hops where we apply this change one hash at a time. The number of intermediate hops is bounded by $\lambda q \leq \lambda^2 \sqrt{n}$.

simulation of IntHyb_6 to adversary $\mathcal{B}$, up to step 3, i.e. when $\mathcal{B}$ returns its answers. At that point, $\mathcal{A}$ returns to the collision-resistance challenger the value $x := (p_1||\ldots||p_{n^{1/4}})$ honestly computed by $\mathcal{A}$ itself, and x' which is $\mathcal{B}$'s answer. This concludes the reduction.

$\text{IntHyb}_6^{\mathcal{A}}(1^\lambda)$:

Preprocessing. Identical to IntHyb_5.

Queries. For $t \in [q]$:

1–3. Identical to IntHyb_5.

4. For each $i \in [\lambda]$:
   a) $(S_1, \ldots, S_{n^{1/4}}) \leftarrow \mathsf{ListDecode}(\widetilde{\mathbf{sk}}'_{\lambda+i})$
   b) For each S_j output by ListDecode, set $p_j \leftarrow \bigoplus_{k \in S_j} \mathrm{DB}[k]$
   c) If $a_{\lambda+i} \neq (p_1 \,||\, \ldots \,||\, p_{n^{1/4}})$, set $y_t \leftarrow \perp$ and $H \leftarrow \perp$ and skip this iteration.

5–6. Identical to IntHyb_5.

Output 0

We conclude the proof by arguing why the probability that IntHyb_6 outputs 1 is negligible for any adversary.

We first note that there is only one possible string that will be accepted by the challenger as a response for each of the 2λ programmed keys. We refer to such string as the *correct* answer. Any adversary that responds with strictly less than $\lambda/2$ incorrect answers will certainly not win, since the majority of the answers used for evaluation will still be correct. Hence, we assume that the adversary cheats in at least $\lambda/2$ answers. But we can easily prove that this yields a negligible winning probability. Let $b \geq \lambda/2$ be the number of wrong answers. The probability that none of these answers is chosen for verification is

$$\binom{2\lambda - b}{\lambda} / \binom{2\lambda}{\lambda} \tag{4.2}$$

Furthermore, the higher b, the lower the winning probability. Indeed, $\binom{2\lambda-(b-1)}{\lambda}/\binom{2\lambda}{\lambda} > \binom{2\lambda-b}{\lambda}/\binom{2\lambda}{\lambda}$. Thus, the overall winning probability cannot be higher than (4.2) with b set to $\lambda/2$:

$$\binom{\frac{3\lambda}{2}}{\lambda} / \binom{2\lambda}{\lambda}$$

which is negligible in λ. This concludes the left integrity proof.

4.3.4 Left Privacy for $\lambda\sqrt{n}$ random, distinct queries

For left server privacy, we start from a key observation: in the definition of left privacy, the adversary is informed on whether the client has accepted their answer or not, but contrary to integrity, there is no need to actually reconstruct the bit y_t. However y_t is still necessary to update the hint table. Hence, our first step is to use integrity to swap y_t with $\mathrm{DB}[x_t]$, for each $t \in [q]$. This enables us to define the first hybrid $\textsc{PrivHyb}_1$, where we entirely omit the construction of y_t.

$\textsc{PrivHyb}_1^{\mathcal{A}}(1^\lambda)$:

Preprocessing.

1. $(\mathsf{st}_{\mathcal{A}}, \mathrm{DB}, q) \leftarrow \mathcal{A}(1^\lambda)$, then draw $(x_1, \ldots, x_q)$ at random without replacement
2. $pp_{\mathsf{H}} \leftarrow \mathsf{HashSetup}(1^\lambda)$, $pp_{\mathsf{VC}} \leftarrow \mathsf{VCSetup}(1^\lambda)$, $\sigma \leftarrow \mathsf{Commit}(pp_{\mathsf{VC}}, \mathrm{DB})$
3. Let $M_1 := 2\lambda\sqrt{n}$, $T \leftarrow ()$. Repeat M_1 times:

   a) $\mathbf{sk} \leftarrow \mathsf{Gen}(1^\lambda)$ b) $p := \bigoplus_{j \in \mathsf{Set}(\mathbf{sk})} \mathrm{DB}[j]$ c) $T \leftarrow T \,||\, \Big((\mathbf{sk}, p)\Big)$
4. Let $M_2 := 3\lambda^2$, $R_{\text{right}} \leftarrow ()$, $R_{\text{left}} \leftarrow ()$. For each $\ell \in [\sqrt{n}]$, repeat M_2 times:
   a) Sample random $r_{\text{right}} \in [n]$ in chunk ℓ. Let $\rho_{\text{right}} := \mathrm{DB}[r_{\text{right}}]$
   b) $R_{\text{right}} \leftarrow R_{\text{right}} \,||\, \Big((r_{\text{right}}, \rho_{\text{right}})\Big)$
   c) Sample random $r_{\text{left}} \in [n]$ in chunk ℓ
   d) $((\rho_{\text{left}}, \pi), \mathsf{st}_{\mathcal{A}}) \leftarrow \mathcal{A}(\mathsf{st}_{\mathcal{A}}, pp_{\mathsf{H}}, pp_{\mathsf{VC}}, r_{\text{left}})$
   e) $b \leftarrow \mathsf{Verify}(pp_{\mathsf{VC}}, \sigma, r_{\text{left}}, \rho_{\text{left}}, \pi)$. If $b = 0$, $H \leftarrow \perp$ and skip step 4. Otherwise, $R_{\text{left}} \leftarrow R_{\text{left}} \,||\, \Big((r_{\text{left}}, \rho_{\text{left}})\Big)$
5. Let $V \leftarrow ()$. Repeat $\lambda^2\sqrt{n}$ times:
   a) $\widetilde{\mathbf{sk}} \leftarrow \mathsf{Gen}(1^\lambda)$, $\ell \leftarrow_{\$} [\sqrt{n}]$, and $\delta \leftarrow_{\$} [\sqrt{n}]$
   b) $\widetilde{\mathbf{sk}}' \leftarrow \mathsf{Program}(\widetilde{\mathbf{sk}}, \ell, \delta)$
   c) $(S_1, \ldots, S_{n^{1/4}}) \leftarrow \mathsf{ListDecode}(\widetilde{\mathbf{sk}}')$
   d) For each S_j output by ListDecode, compute $p_j \leftarrow \bigoplus_{k \in S_j} \mathrm{DB}[k]$
   e) $h \leftarrow \mathsf{Eval}(pp_{\mathsf{H}}, p_1 || \ldots || p_{n^{1/4}})$
   f) $V \leftarrow V \,||\, \Big((\widetilde{\mathbf{sk}}', \ell, \delta, h)\Big)$

$\textsc{PrivHyb}_1^{\mathcal{A}}(1^\lambda)$:

Queries. Set $b_0 \leftarrow \perp$. For $t \in [q]$:

1. $\mathsf{st}_{\mathcal{A}} \leftarrow \mathcal{A}(\mathsf{st}_{\mathcal{A}}, b_{t-1})$
2. If $H = \perp$, set $y_t \leftarrow \perp$, $b_t \leftarrow 0$ and skip this iteration.
3. Let $Q = ()$. For each $i \in [\lambda]$:
   a) Find the first entry $(\mathbf{sk}_i, p_i) \in T$ such that $x_t \in \mathsf{Set}(\mathbf{sk}_i)$. If no such entry exists, set $y_t \leftarrow \perp$, $b_t \leftarrow 0$, $H \leftarrow \perp$ and skip this iteration. Otherwise, let this entry be the s_i-th, and set $T \leftarrow T[s_i \leftarrow \perp]$
   b) Find the first $(r_{\text{right},i}, \rho_{\text{right},i}) \in R_{\text{right}}$ such that $r_{\text{right},i}$ is in the $\mathsf{chunk}(x_t)$-th chunk and remove it from R_{right}. If no such entry exists, set $y_t \leftarrow \perp$, $b_t \leftarrow 0$, $H \leftarrow \perp$ and skip this iteration.
   c) $\mathbf{sk}'_i \leftarrow \mathsf{Program}(\mathbf{sk}_i, \mathsf{chunk}(x_t), \mathsf{offset}(r_{\text{right},i}))$
   d) Update $Q \leftarrow Q \,||\, (\mathbf{sk}'_i)$.
4. For $i \in [\lambda]$:
   a) Take $(\widetilde{\mathbf{sk}}_i, \ell_i, \delta_i, h_i) \in V$ and remove it from V
   b) $\widetilde{\mathbf{sk}}'_i \leftarrow \mathsf{Program}(\widetilde{\mathbf{sk}}_i, \ell_i, \delta_i)$. Update $Q \leftarrow Q \,||\, (\widetilde{\mathbf{sk}}'_i)$.
5. $\tau \leftarrow_{\$} \mathsf{Perm}(2\lambda)$, $(\mathsf{st}_{\mathcal{A}}, (a'_1, \ldots, a'_{2\lambda})) \leftarrow \mathcal{A}(\tau(Q), \mathsf{st}_{\mathcal{A}})$, $(a_1, \ldots, a_{2\lambda}) \leftarrow \tau^{-1}\big((a'_1, \ldots, a'_{2\lambda})\big)$.
6. For each $i \in [\lambda]$: if $\mathsf{Eval}(pp_{\mathsf{H}}, a_{\lambda+i}) \neq h_i$, set $y_t \leftarrow \perp$, $b_t \leftarrow 0$ and $H \leftarrow \perp$ and skip this iteration. Otherwise set $b_t \leftarrow 1$
7. Repeat the following λ times:
   a) $\mathbf{sk}_{\text{new}} \leftarrow \mathsf{Gen}(1^\lambda)$ such that $x_t \in \mathsf{Set}(\mathbf{sk}_{\text{new}})$
   b) Find the first $(r_{\text{left}}, \rho_{\text{left}}) \in R_{\text{left}}$ such that r_{left} is in the $\mathsf{chunk}(x_t)$-th chunk and remove it from R_{left}
   c) $\mathbf{sk}'_{\text{new}} \leftarrow \mathsf{Program}(\mathbf{sk}_{\text{new}}, \mathsf{chunk}(x_t), \mathsf{offset}(r_{\text{left}}))$, and then compute $j_{\text{new}} \leftarrow \mathsf{Recover}(\mathbf{sk}'_{\text{new}}, \mathsf{chunk}(x_t))$
   d) $(S_1, \ldots, S_{n^{1/4}}) \leftarrow \mathsf{ListDecode}(\mathbf{sk}'_{\text{new}})$, $\beta \leftarrow \bigoplus_{i \in S_{j_{\text{new}}}} \text{DB}[i]$
   e) $T \leftarrow T[s_i \leftarrow (\mathbf{sk}_{\text{new}}, \rho_{\text{left}} \oplus \beta \oplus \text{DB}[x_t])]$

Output $\mathcal{A}(\mathsf{st}_{\mathcal{A}}, b_q)$

Now consider $\textsc{PrivHyb}_2$ below. Similar to the integrity proof, we use Lemma 4.2 to simplify the protocol by de-correlating the keys sent to the adversary. The reduction is similar to the integrity proof: the lemma distinguisher $\mathcal{D}$ can perfectly simulate the two privacy games to the privacy adversary, and this is even easier thanks to the absence of the step where we reconstruct y_t.

$\textsc{PrivHyb}_2^{\mathcal{A}}(1^\lambda)$:

Preprocessing. Identical to $\textsc{PrivHyb}_1$, but step 3 is skipped and R_{right}, and R_{left} are not used anymore.

Queries. Set $b_0 \leftarrow \perp$. For $t \in [q]$:

1. $\mathsf{st}_{\mathcal{A}} \leftarrow \mathcal{A}(\mathsf{st}_{\mathcal{A}}, b_{t-1})$
2. If $H = \perp$, set $b_t \leftarrow 0$ and skip this iteration.
3. Let $Q = ()$. For each $i \in [\lambda]$:
   a) Sample $\mathbf{sk}_i \leftarrow \mathsf{Gen}(1^\lambda)$ conditioned on $x_t \in \mathsf{Set}(\mathbf{sk}_i)$.
   b) Sample $r \leftarrow_\$ [\sqrt{n}]$
   c) $\mathbf{sk}_i' \leftarrow \mathsf{Program}(\mathbf{sk}_i, \mathsf{chunk}(x_t), \mathsf{offset}(r))$
   d) Update $Q \leftarrow Q \,||\, (\mathbf{sk}_i')$.
4. For $i \in [\lambda]$:
   a) Take $(\widetilde{\mathbf{sk}}_i, \ell_i, \delta_i, h_i) \in V$ and remove it from V
   b) $\widetilde{\mathbf{sk}}_i' \leftarrow \mathsf{Program}(\widetilde{\mathbf{sk}}_i, \ell_i, \delta_i)$. Update $Q \leftarrow Q \,||\, (\widetilde{\mathbf{sk}}_i')$.
5. $\tau \leftarrow_\$ \mathsf{Perm}(2\lambda)$, $(\mathsf{st}_{\mathcal{A}}, (a_1', \ldots, a_{2\lambda}')) \leftarrow \mathcal{A}(\tau(Q), \mathsf{st}_{\mathcal{A}})$, $(a_1, \ldots, a_{2\lambda}) \leftarrow \tau^{-1}((a_1', \ldots, a_{2\lambda}'))$.
6. For each $i \in [\lambda]$: if $\mathsf{Eval}(pp_{\mathsf{H}}, a_{\lambda+i}) \neq h_i$, set $b_t \leftarrow 0$ and $H \leftarrow \perp$ and skip this iteration. Otherwise set $b_t \leftarrow 1$

Output $\mathcal{A}(\mathsf{st}_{\mathcal{A}}, b_q)$

Now consider the simulator below, which we write in its entirety to emphasize that the queries are not used. In $\textsc{PrivHyb}_2$, the only dependency on x_t is when sampling $\mathbf{sk}_i$ conditioned on $x_t \in \mathsf{Set}(\mathbf{sk}_i)$. However, we can yet again use the private programmability property of the PPPS. This completely removes dependency on x_t, which makes $\textsc{Sim}$ a proper simulator.

This concludes the left privacy proof.

$\textsc{Sim}^{\mathcal{A}}(1^\lambda)$:

1. $(\mathsf{st}_{\mathcal{A}}, \mathrm{DB}, q) \leftarrow \mathcal{A}(1^\lambda)$
2. $pp_{\mathsf{H}} \leftarrow \mathsf{HashSetup}(1^\lambda)$, $pp_{\mathsf{VC}} \leftarrow \mathsf{VCSetup}(1^\lambda)$, $\sigma \leftarrow \mathsf{Commit}(pp_{\mathsf{VC}}, \mathrm{DB})$
3. Let $M_2 := 3\lambda^2$. For each $\ell \in [\sqrt{n}]$, repeat M_2 times:
   a) Sample random $r_{\text{left}} \in [n]$ in chunk ℓ
   b) $((\rho_{\text{left}}, \pi), \mathsf{st}_{\mathcal{A}}) \leftarrow \mathcal{A}(\mathsf{st}_{\mathcal{A}}, pp_{\mathsf{H}}, pp_{\mathsf{VC}}, r_{\text{left}})$
   c) $b \leftarrow \mathsf{Verify}(pp_{\mathsf{VC}}, \sigma, r_{\text{left}}, \rho_{\text{left}}, \pi)$. If $b = 0$, $H \leftarrow \perp$ and skip step 4.
4. Let $V \leftarrow ()$. Repeat $\lambda^2\sqrt{n}$ times:
   a) $\widetilde{\mathbf{sk}} \leftarrow \mathsf{Gen}(1^\lambda)$, $\ell \leftarrow_{\$} [\sqrt{n}]$, and $\delta \leftarrow_{\$} [\sqrt{n}]$
   b) $\widetilde{\mathbf{sk}}' \leftarrow \mathsf{Program}(\widetilde{\mathbf{sk}}, \ell, \delta)$
   c) $(S_1, \ldots, S_{n^{1/4}}) \leftarrow \mathsf{ListDecode}(\widetilde{\mathbf{sk}}')$
   d) For each S_j output by ListDecode, compute $p_j \leftarrow \bigoplus_{k \in S_j} \mathrm{DB}[k]$
   e) $h \leftarrow \mathsf{Eval}(pp_{\mathsf{H}}, p_1 || \ldots || p_{n^{1/4}})$
   f) $V \leftarrow V \,||\, \left((\widetilde{\mathbf{sk}}', \ell, \delta, h)\right)$

Set $b_0 \leftarrow \perp$. For $t \in [q]$:

1. $\mathsf{st}_{\mathcal{A}} \leftarrow \mathcal{A}(\mathsf{st}_{\mathcal{A}}, b_{t-1})$
2. If $H = \perp$, set $b_t \leftarrow 0$ and skip this iteration.
3. Let $Q = ()$. For each $i \in [\lambda]$:
   a) Sample $\mathbf{sk}_i \leftarrow \mathsf{Gen}(1^\lambda)$
   b) Sample $r \leftarrow_{\$} [n]$
   c) $\mathbf{sk}_i' \leftarrow \mathsf{Program}(\mathbf{sk}_i, \mathsf{chunk}(r), \mathsf{offset}(r))$
   d) Update $Q \leftarrow Q \,||\, (\mathbf{sk}_i')$.
4. For $i \in [\lambda]$:
   a) Take $(\widetilde{\mathbf{sk}}_i, \ell_i, \delta_i, h_i) \in V$ and remove it from V
   b) $\widetilde{\mathbf{sk}}_i' \leftarrow \mathsf{Program}(\widetilde{\mathbf{sk}}_i, \ell_i, \delta_i)$. Update $Q \leftarrow Q \,||\, (\widetilde{\mathbf{sk}}_i')$.
5. $\tau \leftarrow_{\$} \mathsf{Perm}(2\lambda)$, $(\mathsf{st}_{\mathcal{A}}, (a_1', \ldots, a_{2\lambda}')) \leftarrow \mathcal{A}(\tau(Q), \mathsf{st}_{\mathcal{A}})$, $(a_1, \ldots, a_{2\lambda}) \leftarrow \tau^{-1}\big((a_1', \ldots, a_{2\lambda}')\big)$.
6. For each $i \in [\lambda]$: if $\mathsf{Eval}(pp_{\mathsf{H}}, a_{\lambda+i}) \neq h_i$, set $b_t \leftarrow 0$ and $H \leftarrow \perp$ and skip this iteration. Otherwise set $b_t \leftarrow 1$

Output $\mathcal{A}(\mathsf{st}_{\mathcal{A}}, b_q)$

4.3.5 Right Privacy for $\lambda\sqrt{n}$ random, distinct queries

Since right privacy assumes an honest-but-curious setting, the proof becomes significantly simpler. We start by observing that since the whole preprocessing phase is executed with no dependency on the queries $x_1, \ldots, x_q$, $\mathsf{view}_{S^{\text{query}}_{\text{right}}}$ is clearly independent of the queries in both RightReal and RightIdeal. In the query phase for query $x \in [n]$, the view of $S^{\text{query}}_{\text{right}}$ in RightReal has the following elements:

- λ programmed keys computed as follows:
  1. $\mathbf{sk}_{\text{new}} \leftarrow \mathsf{Gen}(1^\lambda)$ conditioned on $x \in \mathsf{Set}(\mathbf{sk}_{\text{new}})$
  2. $\mathbf{sk}'_{\text{new}} \leftarrow \mathsf{Program}(\mathbf{sk}_{\text{new}}, \mathsf{chunk}(x), \mathsf{offset}(r_{\text{left}}))$ where r_{left} is a random index sampled in the preprocessing phase (and only seen by the left server).
- Parities obtained with a computation on the DB and the $\mathbf{sk}_{\text{new}}$ keys.

As explained previously, the complexity of the left privacy proof arises from the presence of the hint table, requiring an argument (Lemma 4.2) that decorrelated the programmed keys sent to the left server. Here the reduction to PPPS private programmability does not need any preparatory hybrids: the programmed keys are generated on the fly. After this hop, we already obtain a valid simulator, concluding the right privacy proof.

4.3.6 Upgrading to arbitrary, unbounded queries

As a last step in our proof, we show how to obtain a PIR protocol satisfying Theorem 4.1, starting from our protocol as a black box, as done in [19, 10]:

- To overcome the requirement of *distinct* queries, the client can simply cache the result of a query. Also, notice that since the number of queries is bounded by $\widetilde{\mathcal{O}}_\lambda(\sqrt{n})$, this change does not affect the size of client storage.
- Now, to allow for *any* sequence of queries, it suffices to shuffle the database. However, all parties in the protocol must be aware of how the database has been permuted. To do so, the client can sample a PRP key and send it to both servers in the setup phase.
- The bound on q might seem the most problematic to solve. At the moment, our protocol would need the preprocessing phase to be re-run from scratch after q queries (we call this the *next* preprocessing phase). However, consider the costs of the next preprocessing phase as shown in Section 4.2:
  - $\widetilde{\mathcal{O}}_\lambda(\sqrt{n})$ bandwidth

- $\widetilde{\mathcal{O}}_\lambda(n)$ server time
- $\widetilde{\mathcal{O}}_\lambda(\sqrt{n})$ client time

These costs are low enough that we can amortize them on the current queries: that is, for a given (arbitrary) number q of queries $x_1, \ldots, x_q$, we split the queries into batches of size $\lambda\sqrt{n}$. Then, for each batch, we run the next preprocessing phase while we run the query phases for the current batch of queries. Indeed, the $\widetilde{\mathcal{O}}_\lambda(n)$ server work needed in the preprocessing phase can be split into $\widetilde{\mathcal{O}}_\lambda(\sqrt{n})$ work per query, hence the asymptotics are not worsened. A similar argument holds for bandwidth and client time.

We limit ourselves to this informal description of the new protocol, as the ideas used in it are identical to those in [19, 10], but we will now briefly argue about its correctness, privacy and integrity. Correctness crucially depends on the correctness of the underlying PIR protocol. Hence, a formal argument on its correctness would require a proof that the correctness game from Definition 3.2 is correctly simulated to the underlying protocol. This is guaranteed by the PRP, and by assuming that the underlying PIR protocol is only invoked for new queries. Both privacy and integrity follow from the same reasoning.

Chapter 5

Conclusion

With the protocol described in Chapter 4, we managed to lower the best known theoretical costs of Authenticated PIR with client-side preprocessing. Our contribution pushes Authenticated PIR one step closer to practicality. Indeed, the achieved $\widetilde{\mathcal{O}}_\lambda(n^{1/4})$ bandwidth is low enough to implement it on databases at the exabyte scale, where the communication cost would amount to only a few kilobytes. However, hidden λ factors in the stated complexities, whose optimization we did not address in this work, still need refinement before a practical implementation is possible.

In what follows, we outline a few possible directions for future work in this area.

Future Work

We start by observing that our attacker model could be strengthened by assuming that at most one of the servers is malicious, regardless of its role in the protocol. This notion is stronger than ours, as we only assume the left server to be malicious. Our protocol does not satisfy integrity in this model, since the validity of the messages sent by the right server is not checked. One could argue that trust in the right server is more justifiable, since most of the communication with it takes place in one single communication round in the preprocessing phase, whereas the left server exchanges messages throughout all the queries. However, this is not entirely true: firstly, there is communication with the right server in the query phase as well (to refresh the hint table), and secondly, as explained in Section 4.3.6, for more than $\lambda\sqrt{n}$ queries, the *next* preprocessing phase is actually performed iteratively, in parallel with the current query phase.

Another interesting direction for future work is related to Falk et al.'s [7] work. As discussed in Chapter 1, this paper shows that it is possible to

compile any PIR protocol to an Authenticated PIR protocol by using the underlying PIR protocol as a black-box. However, their definition of PIR does not account for client-side preprocessing protocols. This leaves open the question of whether such a generic transformation is possible for client-side preprocessing schemes.

Lastly, we also highlight the very recent result of Goshal et al. [11]. Despite its practical infeasibility, this work shows that client-side preprocessing PIR is obtainable with polylogarithmic bandwidth. Crucially, this scheme does not provide integrity, and it would be worthwhile to investigate the existence of an Authenticated PIR protocol with similar costs.

Bibliography

[1] Amos Beimel, Yuval Ishai, and Tal Malkin. Reducing the servers' computation in private information retrieval: Pir with preprocessing. *J. Cryptology*, 17:125–151, 2004.

[2] Christian Cachin, Silvio Micali, and Markus Stadler. Computationally private information retrieval with polylogarithmic communication. In *Proceedings of the 17th International Conference on Theory and Application of Cryptographic Techniques*, EUROCRYPT'99, page 402–414, Berlin, Heidelberg, 1999. Springer-Verlag.

[3] B. Chor, O. Goldreich, E. Kushilevitz, and M. Sudan. Private information retrieval. In *Proceedings of IEEE 36th Annual Foundations of Computer Science*, pages 41–50, 1995.

[4] Benny Chor and Niv Gilboa. Computationally private information retrieval (extended abstract). In *Proceedings of the Twenty-Ninth Annual ACM Symposium on Theory of Computing*, STOC '97, page 304–313, New York, NY, USA, 1997. Association for Computing Machinery.

[5] Simone Colombo, Kirill Nikitin, Henry Corrigan-Gibbs, David J. Wu, and Bryan Ford. Authenticated private information retrieval. Cryptology ePrint Archive, Paper 2023/297, 2023.

[6] Henry Corrigan-Gibbs and Dmitry Kogan. Private information retrieval with sublinear online time. Cryptology ePrint Archive, Paper 2019/1075, 2019.

[7] Brett Falk, Pratyush Mishra, and Matan Shtepel. Malicious security for PIR (almost) for free. Cryptology ePrint Archive, Paper 2024/964, 2024.

[8] William Gasarch. A survey on private information retrieval. *Bulletin of the EATCS*, (82):72–107, 2004.

[9] Craig Gentry and Zulfikar Ramzan. Single-database private information retrieval with constant communication rate. In Luís Caires, Giuseppe F. Italiano, Luís Monteiro, Catuscia Palamidessi, and Moti Yung, editors, *Automata, Languages and Programming*, pages 803–815, Berlin, Heidelberg, 2005. Springer Berlin Heidelberg.

[10] Ashrujit Ghoshal, Mingxun Zhou, and Elaine Shi. Efficient pre-processing PIR without public-key cryptography. Cryptology ePrint Archive, Paper 2023/1574, 2023.

[11] Ashrujit Ghoshal, Mingxun Zhou, Elaine Shi, and Bo Peng. Pseudo-random functions with weak programming privacy and applications to private information retrieval. Cryptology ePrint Archive, Paper 2025/300, 2025.

[12] E. Kushilevitz and R. Ostrovsky. Replication is not needed: single database, computationally-private information retrieval. In *Proceedings 38th Annual Symposium on Foundations of Computer Science*, pages 364–373, 1997.

[13] Arthur Lazzaretti and Charalampos Papamanthou. TreePIR: Sublinear-time and polylog-bandwidth private information retrieval from DDH. Cryptology ePrint Archive, Paper 2023/204, 2023.

[14] Wei-Kai Lin, Ethan Mook, and Daniel Wichs. Doubly efficient private information retrieval and fully homomorphic RAM computation from ring LWE. Cryptology ePrint Archive, Paper 2022/1703, 2022.

[15] Ralph C Merkle. A digital signature based on a conventional encryption function. In *Conference on the theory and application of cryptographic techniques*, pages 369–378. Springer, 1987.

[16] Rafail Ostrovsky and William E. Skeith III. A survey of single database PIR: Techniques and applications. Cryptology ePrint Archive, Paper 2007/059, 2007.

[17] Elaine Shi, Waqar Aqeel, Balakrishnan Chandrasekaran, and Bruce Maggs. Puncturable pseudorandom sets and private information retrieval with near-optimal online bandwidth and time. Cryptology ePrint Archive, Paper 2020/1592, 2020.

[18] Yinghao Wang, Xuanming Liu, Jiawen Zhang, Jian Liu, and Xiaohu Yang. Crust: Verifiable and efficient private information retrieval with sublinear online time. Cryptology ePrint Archive, Paper 2023/1607, 2023.

[19] Mingxun Zhou, Andrew Park, Elaine Shi, and Wenting Zheng. Piano: Extremely simple, single-server PIR with sublinear server computation. Cryptology ePrint Archive, Paper 2023/452, 2023.

Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

Declaration of originality

The signed declaration of originality is a component of every written paper or thesis authored during the course of studies. **In consultation with the supervisor**, one of the following two options must be selected:

☒ I hereby declare that I authored the work in question independently, i.e. that no one helped me to author it. Suggestions from the supervisor regarding language and content are excepted. I used no generative artificial intelligence technologies[1].

☐ I hereby declare that I authored the work in question independently. In doing so I only used the authorised aids, which included suggestions from the supervisor regarding language and content and generative artificial intelligence technologies. The use of the latter and the respective source declarations proceeded in consultation with the supervisor.

Title of paper or thesis:

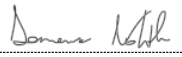
Improving the Asymptotics of Authenticated Private Information Retrieval with Client-Side Preprocessing

Authored by:
If the work was compiled in a group, the names of all authors are required.

Last name(s):	**First name(s):**
Nobile	Domenico

With my signature I confirm the following:
- I have adhered to the rules set out in the Citation Guidelines.
- I have documented all methods, data and processes truthfully and fully.
- I have mentioned all persons who were significant facilitators of the work.

I am aware that the work may be screened electronically for originality.

Place, date	**Signature(s)**
Zürich, 2025/12/04	

If the work was compiled in a group, the names of all authors are required. Through their signatures they vouch jointly for the entire content of the written work.

[1] For further information please consult the ETH Zurich websites, e.g. https://ethz.ch/en/the-eth-zurich/education/ai-in-education.html and https://library.ethz.ch/en/researching-and-publishing/scientific-writing-at-eth-zurich.html (subject to change).